
Chapter 12: Query Processing



Chapter 12: Query Processing

- ▶ In this chapter, we will discuss the range of activities involved in *extracting data* from a database. These activities are collectively called **query processing**.
- ▶ The activities include
 - ▶ *Translation of queries in high-level database languages into expressions* that can be used at the physical level of the file system
 - ▶ *A variety of query-optimizing transformations*
 - ▶ *Actual evaluation of queries*



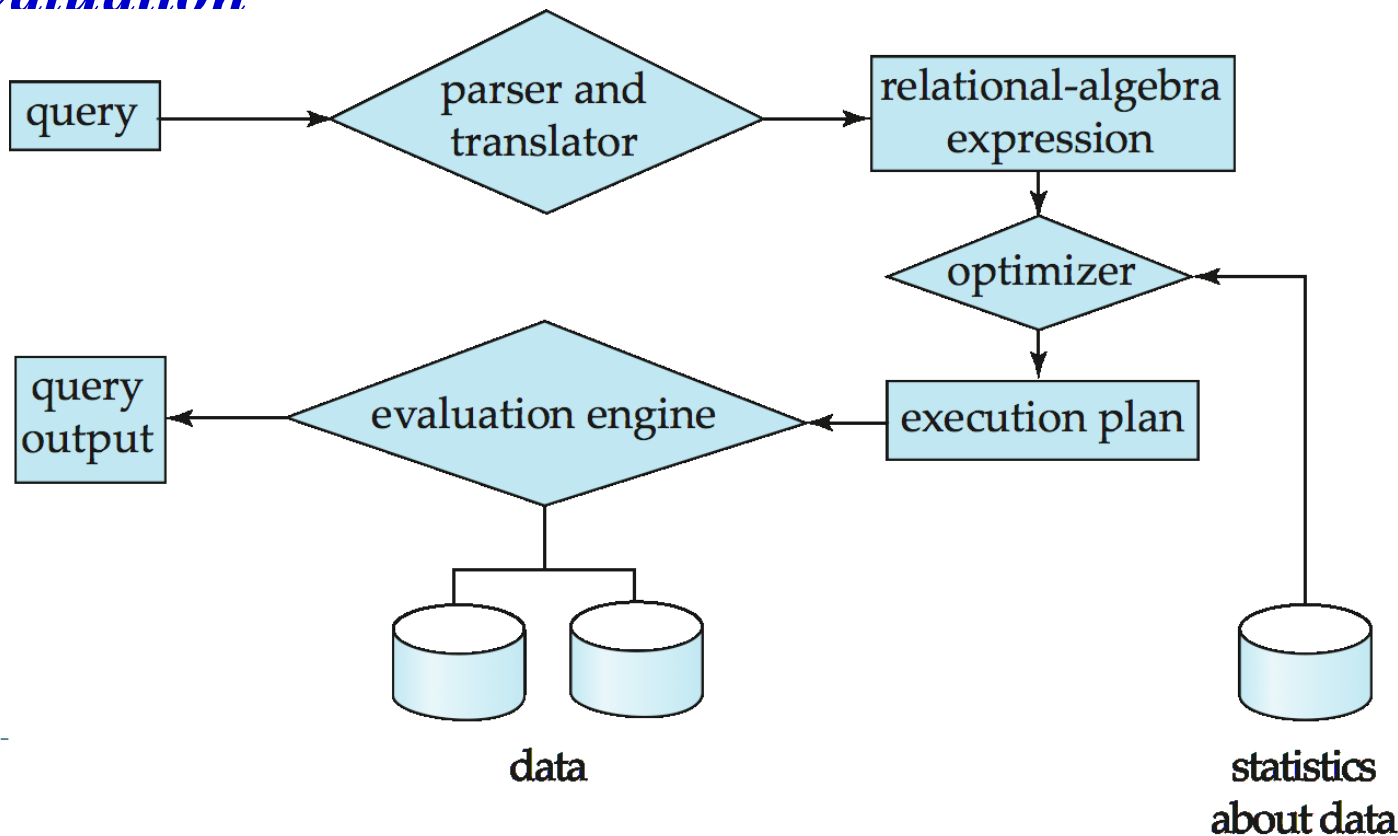
Chapter 12: Query Processing

- ▶ **Overview**
- ▶ **Measures of Query Cost**
- ▶ **Selection Operation**
- ▶ **Sorting**
- ▶ **Join Operation**
- ▶ **Other Operations**
- ▶ **Evaluation of Expressions**

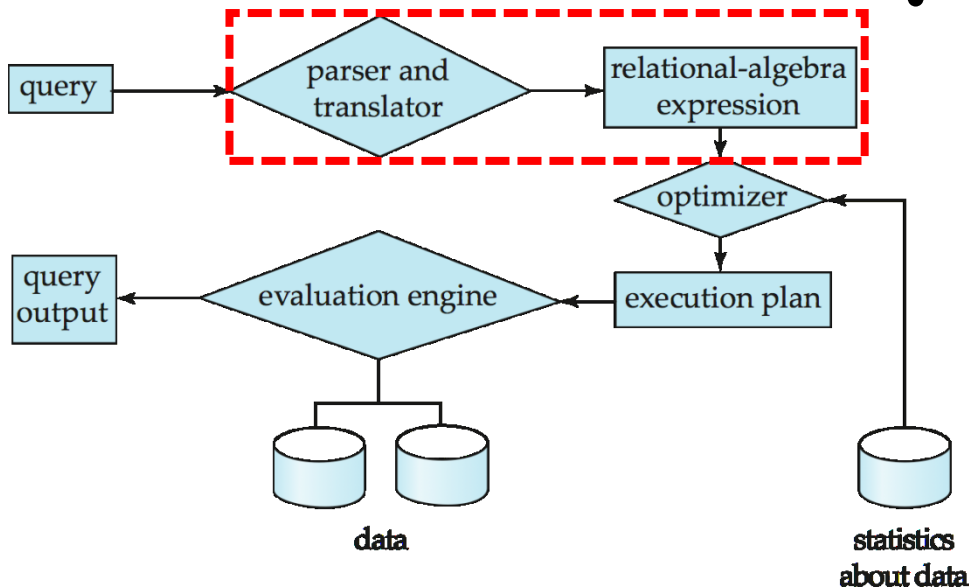


Overview

- ▶ Basic steps in query processing
 - ▶ *Parsing and translation*
 - ▶ *Optimization*
 - ▶ *Evaluation*



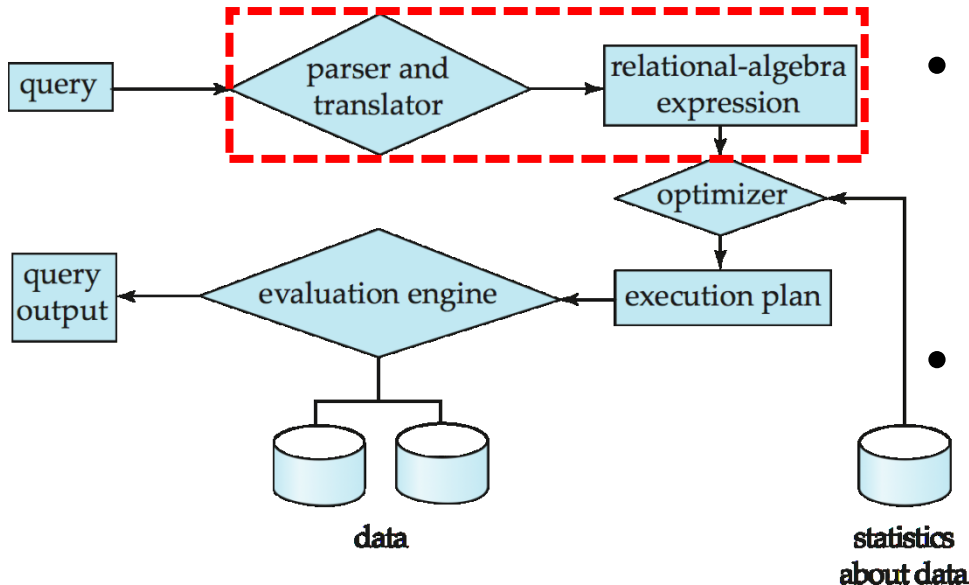
Query Processing



- To start a query, the system must **translate the query into a usable form (internal form)**
 - Checks the syntax of the user's query
 - Verifies that the relation names appearing in the query are names of the relations in the database, and so on.
- The system **constructs a parse-tree representation of the query** which it then translates into a **relational-algebra expression**.
- If the query was expressed in terms of a **view**, the translation phase also replaces all uses of the view by the **relational-algebra expression** that defines the view.



Query Processing



- Example query

select salary
from instructor
where salary < 75000

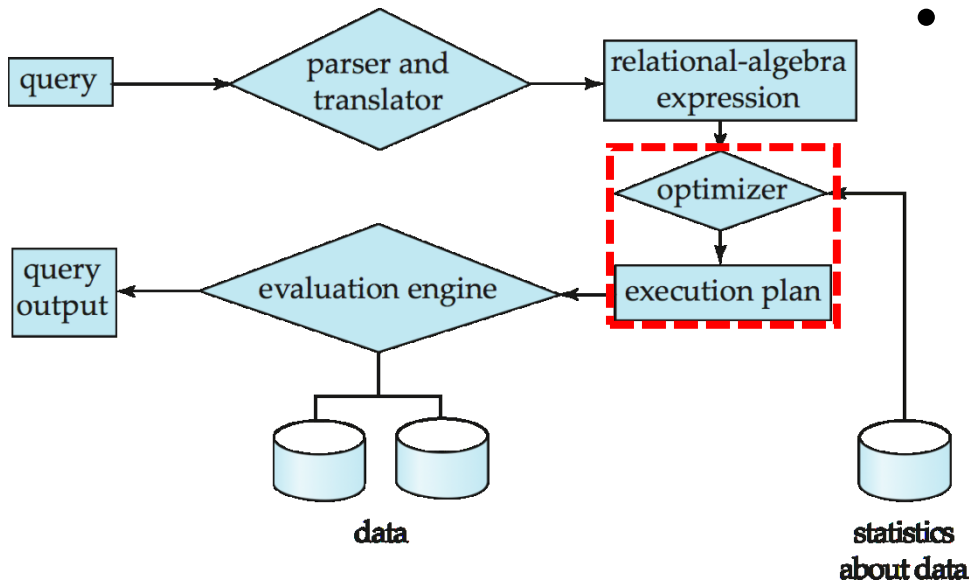
- Alternative expressions

$\sigma_{\text{salary} < 75000}(\Pi_{\text{salary}}(\text{instructor}))$
 $\Pi_{\text{salary}}(\sigma_{\text{salary} < 75000}(\text{instructor}))$

- A query could be expressed in several different ways, and each SQL query can itself be **translated into a relational-algebra expression in one of several ways.**



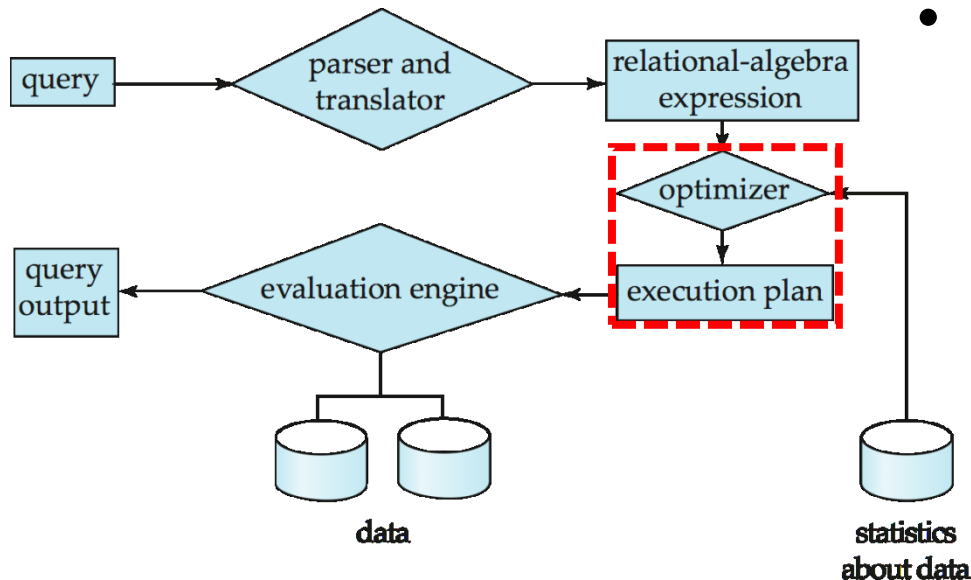
Query Processing



- For example
 - To implement the preceding selection, we can **search every tuple in instructor to find tuples with salary less than 75000.**
 - If a **B^+ -tree index is available on the attribute salary**, we can use the index instead to locate the tuples.
- We can **execute each relational-algebra operation by one of several different algorithms.**



Query Processing

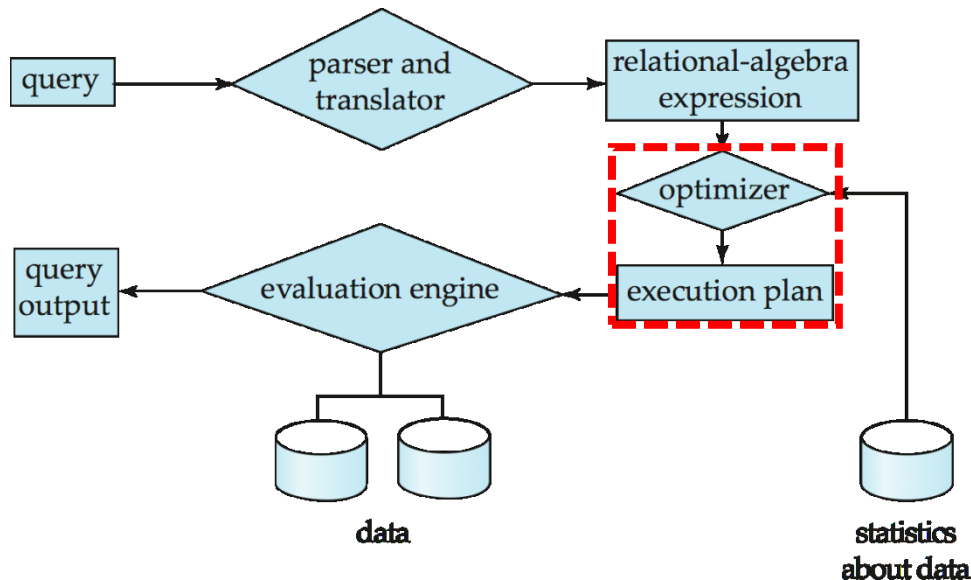


- To **specify fully how to evaluate a query**, we need
 - not only to provide the **relational-algebra expression**
 - but also to annotate it with **instructions specifying how to evaluate each operation**

- **Annotations** may state the algorithm to be used for a **specific operation**, or the **particular index or indices** to use
- A **relational-algebra operation annotated with instructions** on how to evaluate it is called an **evaluation primitive**



Query Processing

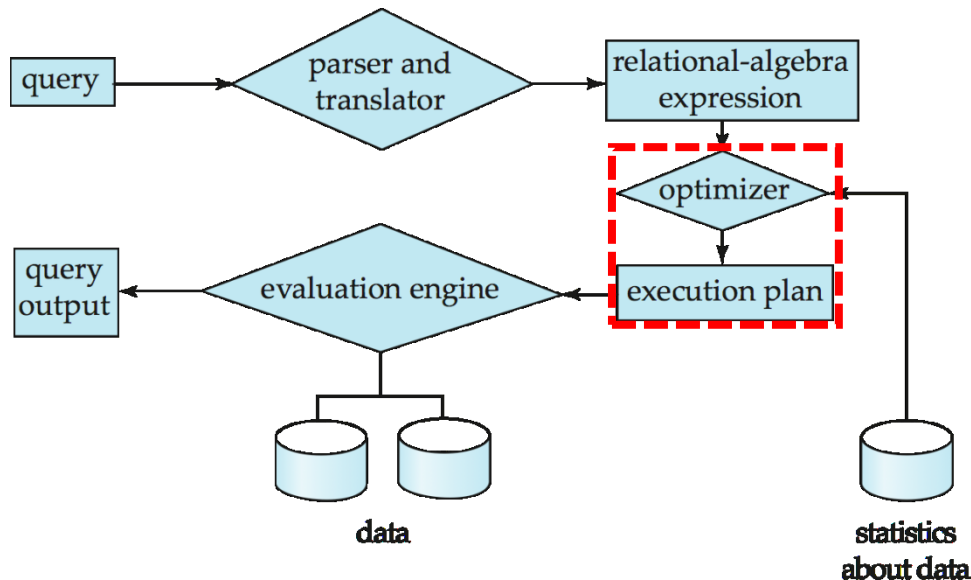


π_{salary}
|
 $\sigma_{salary < 75000; \text{ use index 1}}$
|
instructor

- A sequence of evaluation primitives for a query is called an **query-execution plan** or **query-evaluation plan**.



Query Processing

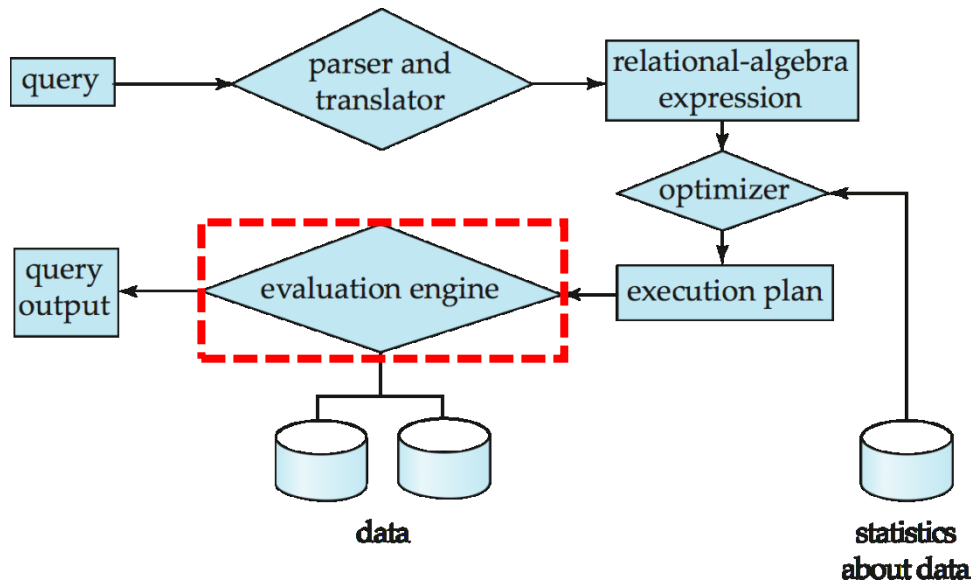


π_{salary}
|
 $\sigma_{salary < 75000; \text{ use index 1}}$
|
instructor

- The **different evaluation plans** for a given query can have **different costs**
- The system will construct a **query evaluation plan** that **minimizes the cost of query evaluation**; this task is called **query optimization**



Query Processing



π_{salary}
|
 $\sigma_{salary < 75000; \text{ use index 1}}$
|
instructor

- The **query-execution engine takes** a query-evaluation plan, **executes** that plan, and **returns** the answers to the query.



Measures of Query Cost

- ▶ To choose the best plan, we must *get the cost of every available query-evaluation plan*. Many factors contribute to time cost
 - ▶ *Disk accesses*
 - ▶ *CPU time to execute a query*
 - ▶ *Cost of communication in a distributed or parallel database system*

Measures of Query Cost (Cont.)

- ▶ In large database systems, the *cost to access data from disk* is usually the most important cost.
- ▶ *Disk accesses are slow* compared to in-memory operations.
- ▶ *CPU speeds* have been improving much faster than have disk speeds.
- ▶ The time spent in disk activity will continue to dominate the total time to execute a query.
- ▶ The CPU time taken for a task is *harder to estimate* since it depends on low-level details of the execution code.
- ▶ So in this book we *ignore CPU costs* and *use only disk-access costs* to measure the cost of a query-evaluation plan.



Measures of Query Cost (Cont.)

- ▶ We can use the *number of block transfers from disk* and the *number of seeks* to estimate the **cost of a query-evaluation plan**.
- ▶ t_T – *average time to transfer a block of data* (typical values for high-end disks: 4 milliseconds)
- ▶ t_S – *an average block-access time* (disk seek time plus rotational latency) (typical values for high-end disks: 0.1 milliseconds)
- ▶ If an operation need *b block transfers* and *S seeks*, the cost will be $b * t_T + S * t_S$.



Measures of Query Cost (Cont.)

- ▶ Cost estimates can be further refined by distinguishing *block reads* from *block writes* (typically about *twice* as expensive as reads; but ignored for simplicity)
- ▶ The cost of *writing the final result of an operation back to disk* is not included in the cost.
- ▶ The costs of all the algorithms that we consider depend on the *size of the buffer in main memory*.
 - ▶ In the *best case*, all data can be read into the buffers.
 - ▶ In the *worst case*, buffer can hold only a few blocks of data—approximately one block per relation (in cost estimates, we generally *assume the worst case*)



Measures of Query Cost (Cont.)

- ▶ Although we assume that *data must be read from disk initially*, it is possible that a block that is accessed is *already present* in the in-memory buffer (ignored for simplicity)
- ▶ The *actual disk-access cost* during the execution of a plan may be less than the estimated cost.
- ▶ Assuming no other activity is going on in the computer, the *response time for a query-evaluation plan* (that is, the wall-clock time required to execute the plan) would *account for all these costs* and could be used *as a measure of the cost of the plan*.



Measures of Query Cost (Cont.)

- ▶ It is *very hard to estimate without actually executing the plan*
- ▶ The response time depends on the *contents of the buffer* when the query begins execution. *This information is not available* when the query is optimized, and is hard to account for even if it were available.
- ▶ In a system with multiple disks, the response time depends on *how accesses are distributed among disks*. *It is hard to estimate* without detailed knowledge of data layout on disk.



Measures of Query Cost (Cont.)

- ▶ Interestingly, a plan may get a better response time at the cost of *extra resource consumption*.
- ▶ A system of multiple disks: **plan A** (requires extra disk reads, but performs the reads in parallel across multiple disks) and **plan B** (has fewer disk reads, but from only one disk)
- ▶ *A* may finish faster than *B*.
- ▶ However, for many instances of a query, using plan *A* run concurrently, the overall response time may be more than using plan *B*, since plan *A* generates more load on the disks.
- ▶ As a result, instead of trying to minimize the response time, optimizers generally *try to minimize the total resource consumption of a query plan*.



Selection Operation: Using File Scan

- ▶ In query processing, the **file scan** is the *lowest-level operator* in query processing.
- ▶ File scans are *search algorithms* that *locate and retrieve records* that *fulfill a selection condition*.



Selection Operation: Using File Scan (Cont.)

- ▶ Consider a **selection operation** on a relation whose *tuples are stored together in one file*. The most straightforward way to perform a selection is as follows
- ▶ **A1 (linear search)**: Scan each file block and test all records to see whether they satisfy the selection condition
 - ▶ Assuming blocks are *stored contiguously*, an *initial seek* is required to access the first block of the file, then the blocks will be transferred sequentially.
 - ▶ If the file are not stored contiguously, extra seeks may be required. But, we ignore this effect for simplicity.
 - ▶ **Cost** = $b_r * t_T + 1 * t_s$
 - ▶ **Note**: b_r denotes the number of blocks in the file



Selection Operation: Using File Scan (Cont.)

- ▶ Consider a **selection operation** on a relation whose *tuples are stored together in one file*. The most straightforward way to perform a selection is as follows
- ▶ **A1 (linear search)**: Scan each file block and test all records to see whether they satisfy the selection condition
 - ▶ Linear search use *equality comparison on Key*.
 - ▶ At most one record satisfies condition. So scan processing can be *terminated* as soon as the required record is found.
 - ▶ In the *worst case*, b_r blocks transfers are still required.
 - ▶ **Average cost** = $(b_r/2) * t_T + 1 * t_s$
 - ▶ **Note**: b_r denotes the number of blocks in the file



Selection Operation: Using File Scan (Cont.)

- ▶ Consider a **selection operation** on a relation whose *tuples are stored together in one file*. The most straightforward way to perform a selection is as follows
- ▶ **A1 (linear search)**: Scan each file block and test all records to see whether they satisfy the selection condition
 - ▶ The *advantage* is that linear search can be *applied to any file*, regardless of
 - ▶ *ordering of records in the file*
 - ▶ *availability of indices*
 - ▶ *nature of selection operation*
 - ▶ However, it is often *slower than other algorithms*



Selection Operation: Using Indices

- ▶ **Index structures** are referred to as *access paths*, since they provide a path through which data can be located and accessed.
- ▶ Search algorithms that use an index is called *index scan*.
- ▶ Search algorithms that use an index are
 - ▶ **A2 (primary index, equality on key)**
 - ▶ **A3 (primary index, equality on nonkey)**
 - ▶ **A4 (secondary index, equality)**



Selection Operation: Using Indices (Cont.)

- ▶ **A2 (primary index, equality on key)**: For an equality comparison on a key attribute with a primary index, use the index to retrieve a single record that satisfies the corresponding equality condition.
- ▶ *Primary B⁺-tree index: Equality on Key*
- ▶ Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
- ▶ *Cost = $h_i * (t_T + t_S) + (t_S + t_T) * 1$* , where h_i denotes the height of the B⁺-tree index.



Selection Operation: Using Indices (Cont.)

- ▶ **A3 (primary index, equality on nonkey)**: Retrieve multiple records by using a primary index when the selection condition specifies an equality comparison on a nonkey attribute, A .
 - ▶ The only difference from the previous case is that *multiple records may need to be fetched*.
 - ▶ However, *the records must be stored consecutively* in the file since the file is sorted on the search key.



Selection Operation: Using Indices (Cont.)

- ▶ **A3 (primary index, equality on nonkey)**: Retrieve multiple records by using a primary index when the selection condition specifies an equality comparison on a nonkey attribute, A .
 - ▶ *Primary B^+ -tree index: Equality on Nonkey*
 - ▶ One seek for each level of the tree, one seek for the first block.
 - ▶ These blocks are *leaf blocks* assumed to be *stored sequentially* (since it is a primary index) and don't require additional seeks.
 - ▶ *Cost* = $h_i * (t_T + t_S) + t_T * b$, where b denotes number of blocks containing matching records.

Selection Operation: Using Indices (Cont.)

- ▶ **A4 (secondary index, equality)**. Selections specifying an *equality condition can use a secondary index*.
 - ▶ *If the equality condition is on a key*, this strategy can retrieve *a single record*.
 - ▶ The time cost in this case is the *same* as that for a primary index (case **A2**).
 - ▶ *Secondary B⁺-tree Index: Equality on Key*
 - ▶ *Cost = $(h_i + 1) * (t_T + t_S)$*



Selection Operation: Using Indices (Cont.)

- ▶ **A4 (secondary index, equality).** Selections specifying an *equality condition can use a secondary index*.
- ▶ *If the indexing field is not a key, multiple records* may be retrieved.
- ▶ Each record may be resident on a different block, which may result in one I/O operation per retrieved record, with each I/O operation requiring a seek and a block transfer.
- ▶ *Secondary B⁺-tree Index: Equality on NonKey*
- ▶ n is the number of records fetched. Each record may be on a different block, requiring a seek per record.
- ▶ *Cost = $(h_i + n) * (t_T + t_S)$* (very expensive when n is large)

Selection Operation: Involving Comparisons

- ▶ Considering a selection of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$, We can implement the selection either by using *linear search* or by using *indices* in one of the following ways
 - ▶ **A5 (primary index, comparison)**. A primary ordered index (e.g., a primary B⁺-tree index) can be used when the selection condition is a comparison.
 - ▶ $\sigma_{A \geq v}(r)$
 - ▶ For $A \geq v$: find the *first tuple* in the file that has a value of $A = v$; file scan starting from that tuple up to the end of the file returns all tuples that satisfy the condition.
 - ▶ For $A > v$: file scan starts with the *first tuple* such that $A > v$.
 - ▶ $Cost = h_i * (t_T + t_S) + t_T * b$ (A₃)

Selection Operation: Involving Comparisons

- ▶ Considering a selection of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$, We can implement the selection either by using *linear search* or by using *indices* in one of the following ways
 - ▶ **A5 (primary index, comparison)**. A primary ordered index (e.g., a primary B⁺-tree index) can be used when the selection condition is a comparison.
 - ▶ $\sigma_{A \leq v}(r)$ (*an index lookup is not required*)
 - ▶ For $A < v$: use a simple file scan starting from the *beginning of the file*, and continuing up to (but not including) the first tuple with attribute $A = v$.
 - ▶ For $A \leq v$: the case of $A \leq v$ is similar, but including the tuple with attribute $A = v$.



Selection Operation: Involving Comparisons

- ▶ Considering a selection of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$, We can implement the selection either by using *linear search* or by using *indices* in one of the following ways
 - ▶ **A6 (secondary index, comparison)**
 - ▶ We can use a *secondary ordered index* to guide retrieval for comparison conditions involving $<$, $\leq$, $\geq$, or $>$.
 - ▶ $\sigma_{A \leq v}(r)$, for $<$ and $\leq$: the lowest-level index blocks are scanned, *from the smallest value up to v* .
 - ▶ $\sigma_{A \geq v}(r)$, for $>$ and $\geq$: *from v up to the maximum value*.



Selection Operation: Involving Comparisons

- ▶ Considering a selection of the form $\sigma_{A \leq v}(r)$ or $\sigma_{A \geq v}(r)$, We can implement the selection either by using *linear search* or by using *indices* in one of the following ways
 - ▶ **A6 (secondary index, comparison)**
 - ▶ The *secondary index provides pointers to the records*, and we have to fetch the actual records by using the pointers.
 - ▶ *Each record* fetched requires *an I/O operation*, since consecutive records may be on *different disk blocks*.
 - Each I/O operation requires *a disk seek* and *a block transfer*.
 - For the large number of retrieved records, using the secondary index may be even more expensive than using linear search.
 - So, the secondary index should be used *only if very few records* are selected.

Implementation of Complex Selections

- ▶ We have considered only simple selection conditions of the form $A \text{ op } B$ (op is an *equality* or *comparison* operation).
- ▶ Now considering more complex selection predicates.
 - ▶ **Conjunction** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
 - ▶ **Disjunction** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$
 - ▶ **Negation** $\sigma_{\neg \theta}(r)$ (the set of tuples of r for which the condition evaluates to false)



Implementation of Complex Selections (Cont.)

- ▶ We can use one of the following algorithms to implement a selection operation involving *either a conjunction or a disjunction* of simple conditions
 - ▶ **A7** (*conjunctive selection using one index*)
 - ▶ **A8** (*conjunctive selection using composite index*)
 - ▶ **A9** (*conjunctive selection by intersection of identifiers*)
 - ▶ **A10** (*disjunctive selection by union of identifiers*)



Implementation of Complex Selections (Cont.)

▶ **A7 (conjunctive selection using one index)**

- ▶ First, determine whether *an access path is available for an attribute in one of the simple conditions*.
- ▶ If one is, *retrieve records* satisfying that condition (using one of the selection algorithms **A2 ~ A6**).
- ▶ In the *memory buffer*, testing whether or not each retrieved records satisfies the *remaining simple conditions* .
- ▶ **To reduce the cost**: we choose an θ_i and one of algorithms **A1 ~ A6** for which the combination results in *the least cost* for $\theta_i(r)$.
- ▶ The cost of algorithm **A7** is given by the cost of the chosen algorithm.

Implementation of Complex Selections (Cont.)

▶ **A8 (conjunctive selection using composite index)**

- ▶ An appropriate *composite index* (that is, *an index on multiple attributes*) may be available for some conjunctive selections.
- ▶ If the selection specifies an *equality condition on two or more attributes*, and a *composite index* exists on these combined attribute fields, then the index can be searched directly.
- ▶ The *type of index* determines which of algorithms **A2, A3, or A4** will be used.



Implementation of Complex Selections (Cont.)

- ▶ **A9 (conjunctive selection by intersection of identifiers)**
 - ▶ Involve the use of *record pointers* or *record identifiers* to implement conjunctive selection operations.
 - ▶ This algorithm requires *indices with record pointers*, on the fields involved in the *individual conditions*.
 - ▶ The algorithm *scans each index for pointers to tuples* that satisfy an *individual condition*.
 - ▶ The *intersection of all the retrieved pointers* is the set of pointers to tuples that satisfy the conjunctive condition.
 - ▶ If indices are not available on all the individual conditions, then the algorithm *tests the retrieved records against the remaining conditions*.

Implementation of Complex Selections (Cont.)

- ▶ **A9 (conjunctive selection by intersection of identifiers)**
 - ▶ The cost of algorithm **A9**
 - ▶ The costs of the *individual index scans*.
 - ▶ The cost of *retrieving the records in the intersection* of the retrieved lists of pointers.
 - ▶ The cost can be reduced by *sorting the list of pointers* and *retrieving records in the sorted order*.
 - ▶ Thereby, all pointers to records in a block come together, hence *all selected records in the block* can be retrieved using *a single I/O operation*.
 - ▶ And *blocks are read in sorted order*, minimizing *disk-arm movement*.

Implementation of Complex Selections (Cont.)

- ▶ **A10 (disjunctive selection by union of identifiers)**
 - ▶ If *access paths* are available on *all the conditions* of a disjunctive selection, *each index* is scanned for pointers to tuples that satisfy the individual condition.
 - ▶ The *union* of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition.
 - ▶ Then fetch records from file.
 - ▶ If *even one* of the conditions *does NOT have an access path*, we have to perform a *linear scan of the relation* to find tuples that satisfy the condition.



Implementation of Complex Selections (Cont.)

- ▶ **Negation:** $\sigma_{\neg\theta}(r)$
 - ▶ Use *linear scan* on file
 - ▶ If *very few records* satisfy $\neg\theta$, and an *index* is applicable to θ , find satisfying records using index and fetch from file
 - ▶ See Practice Exercises 12.6 for examples



Sorting

- ▶ **Sorting** plays an important role in DB systems
 - ▶ *SQL queries* can *specify that the output be sorted*.
 - ▶ Several of the *relational operations can be implemented efficiently* (e.g. join) if the *input relations are first sorted* (equally important for *query processing*)
- ▶ We may build *an index on the sorted key*, and then use the index to read the relation in sorted order. However, such a process orders the relation only *logically*, through an index, rather than *physically*.
- ▶ It may lead to one disk block access for each record.
- ▶ So it may be desirable to *order the records physically*.

Sorting (Cont.)

- ▶ For relations that fit entirely in *main memory*, standard sorting techniques such as *quick-sort* can be used.
- ▶ For relations that are bigger than memory, **external sort-merge** is a good choice.



External Sort-Merge Algorithm

- ▶ The *most commonly used technique for external sorting is the external sort–merge*.
- ▶ **First stage: Create sorted runs (归并段)**. Each run is sorted but contains only some records of the relation.

M: the number of blocks in memory available for sorting

N: the total number of runs

```
i = 0;  
Repeat  
    read M blocks of the relation, or the  
    rest of the relation, whichever is smaller;  
    sort the in-memory part of the relation;  
    write the sorted data to run file  $R_i$ ;  
     $i = i + 1$ ;  
until the end of the relation
```

External Sort-Merge Algorithm (Cont.)

- ▶ **Second stage: Merge the runs.** We assume (for now) that the total number of runs $N < M$.
- ▶ We can *allocate one block to each run* and have *space left to hold one block of output*. The merge stage operates as:

read one block of each of the N files R_i into a buffer block in memory;

Repeat

choose the *first tuple* (in sort order) **among all buffer blocks**;

write the tuple to the output, and **delete** it from the buffer block;

if the buffer block of any run R_i is empty and **not end-of-file** (R_i)

then read the *next block* of R_i into the buffer block;

until all input buffer pages are empty

External Sort-Merge Algorithm (Cont.)

- ▶ The *output of the merge stage* is the **sorted relation**.
- ▶ The output file is *buffered* to *reduce the number of disk write operations*.
- ▶ The above algorithm merges *N runs*, so it is called an **N-way merge**.



External Sort-Merge Algorithm (Cont.)

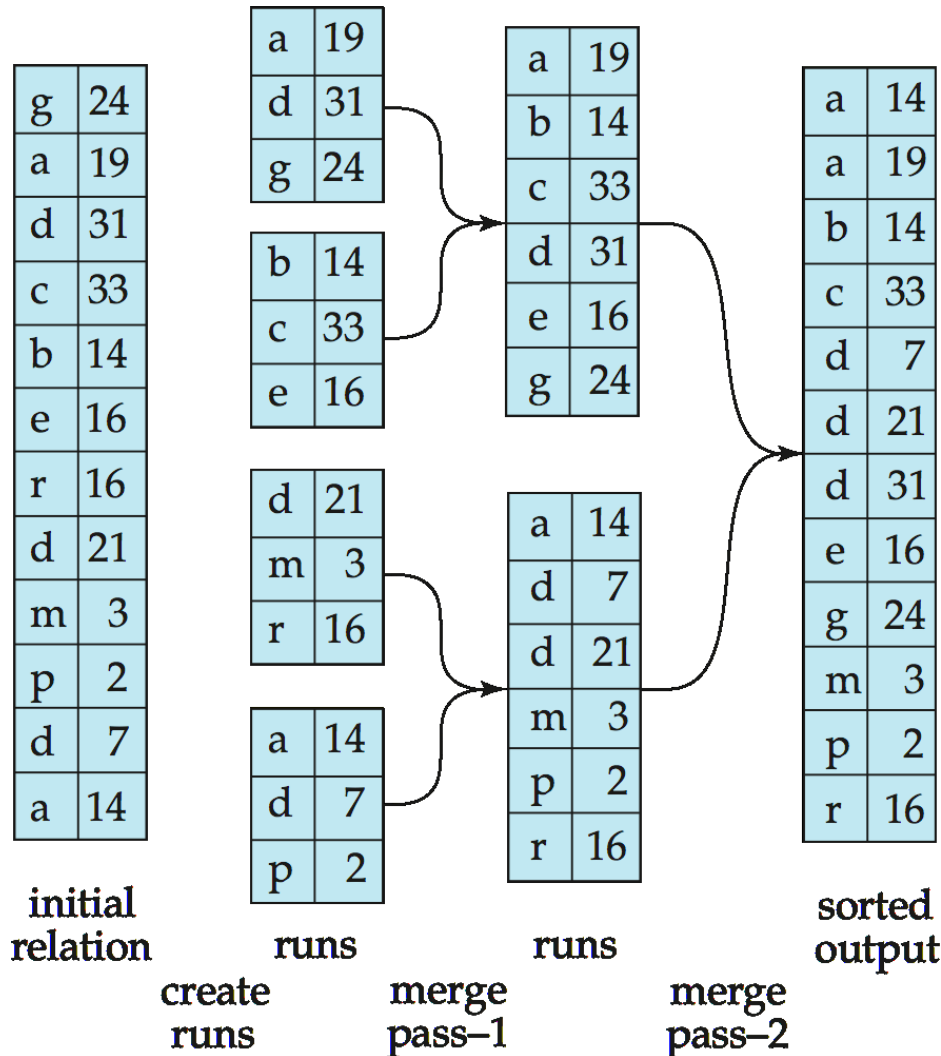
- ▶ In general, if the relation is much larger than memory
 - ▶ In the first stage, may generate M or more runs, that is $N \geq M$.
 - ▶ It is impossible to allocate a block for each run during the merge stage.
 - ▶ In this case, the **merge operation proceeds in multiple passes**.
 - ▶ Since there is enough memory for $M-1$ input buffer blocks, each merge can take $M-1$ runs as input.



External Sort-Merge Algorithm (Cont.)

- ▶ In above case, the **initial pass functions** in this way: merges the *first $M-1$ runs* to get *a single run for the next pass*; then, merges the *next $M-1$ runs* similarly, and so on, until it has processed *all the initial runs*.
- ▶ At this point, the number of runs has been *reduced by a factor of $M-1$* (i.e. $1/(M-1)$).
- ▶ If this reduced number of runs is still greater than or equal to M , another pass is made.
- ▶ The passes repeat as many times as required, until the *number of runs is less than M* .
- ▶ Each pass reduces the number of runs by a factor of $M-1$.

External Sorting Using Sort-Merge: Example

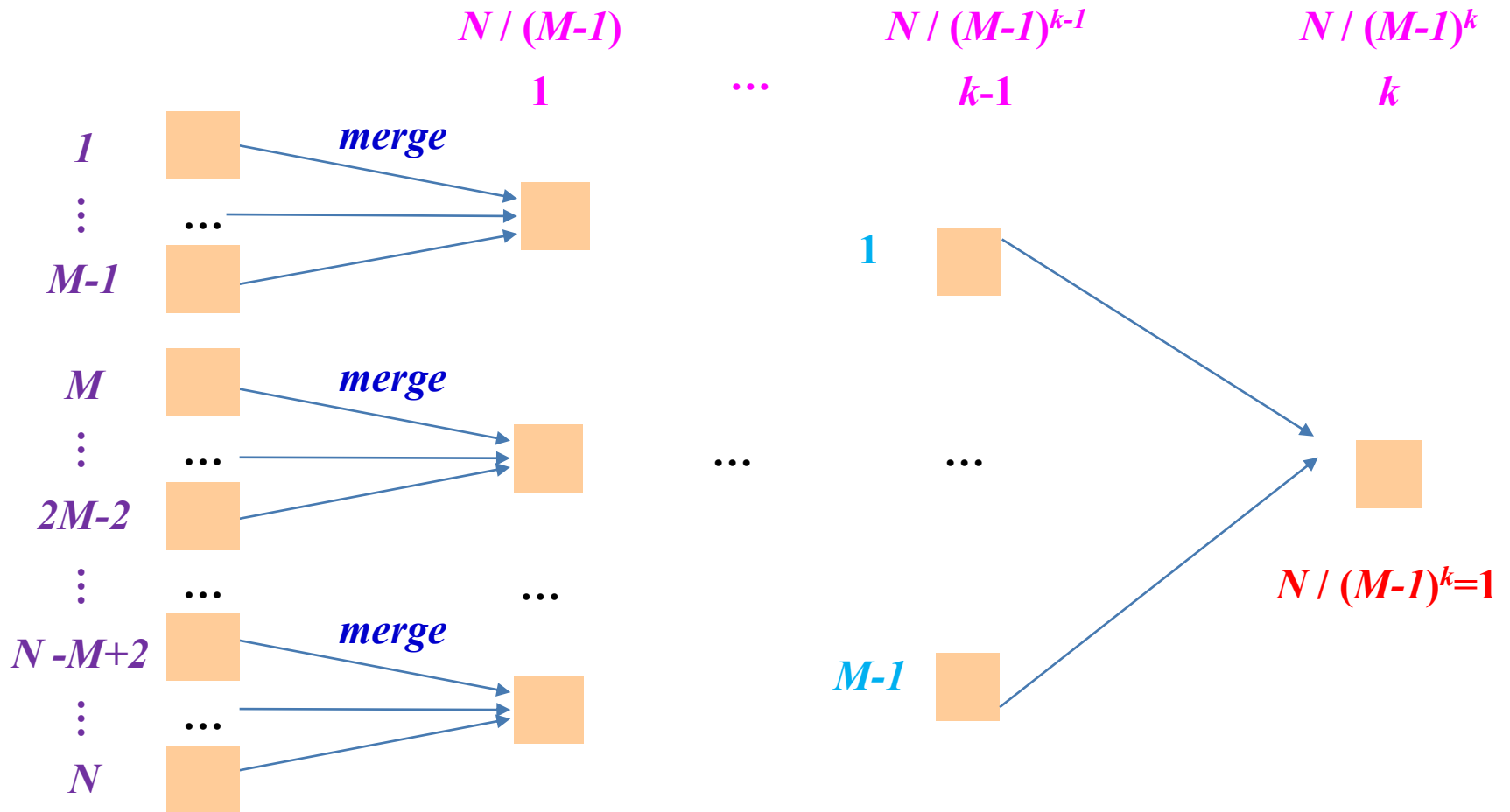


Assumption

- **only one tuple fits in a block** ($f_r = 1$)
- **memory holds 3 blocks**
 - 2 for input
 - 1 for output



External Sorting Using Sort-Merge: Example



b_r : the number of blocks containing records of r

M : the number of blocks in memory available

N : the number of runs, the value is $\lceil b_r / M \rceil$.

External Merge Sort: Cost analysis

- ▶ **Cost of transfers:** (b_r denotes the *number of blocks containing records of r*)
 - ▶ The first stage *reads every block of the relation and writes them out again*, giving a total of $2b_r$ *block transfers*.
 - ▶ The *initial number of runs* is $\lceil b_r / M \rceil$ (i.e. N).
 - ▶ Since the number of runs decreases by a factor of $M-1$ in each merge pass, the *total number of merge passes* required $\lceil \log_{M-1}(b_r/M) \rceil$.

$$N / (M-1)^{k=1} \Rightarrow \lceil b_r / M \rceil / (M-1)^{k=1} \Rightarrow k = \lceil \log_{M-1}(b_r/M) \rceil$$

External Merge Sort: Cost analysis

- ▶ **Cost of transfers:** (b_r denotes the *number of blocks containing records of r*)
- ▶ *Each of these passes reads every block of the relation once and writes it out once* (i.e. $2b_r * \lceil \log_{M-1}(b_r/M) \rceil$), with *two exceptions*:
 - ▶ First, the *final pass* can produce the sorted output without writing its result to disk.
 - ▶ Second, there may be *runs that are not read in or written out during a pass*.
 - E.g.: M runs to be merged in a pass, $M-1$ are read in and merged, and one run is not accessed during the pass.
 - This can be *ignored* since the savings is relatively small.

External Merge Sort: Cost analysis

- ▶ **Cost of transfers:** (b_r denotes the *number of blocks containing records of r*)
- ▶ The *total number of block transfers* for external sorting of the relation is

$$2b_r + (2 \lceil \log_{M-1}(b_r/M) \rceil - 1) * b_r$$



$$b_r (2 \lceil \log_{M-1}(b_r/M) \rceil + 1)$$



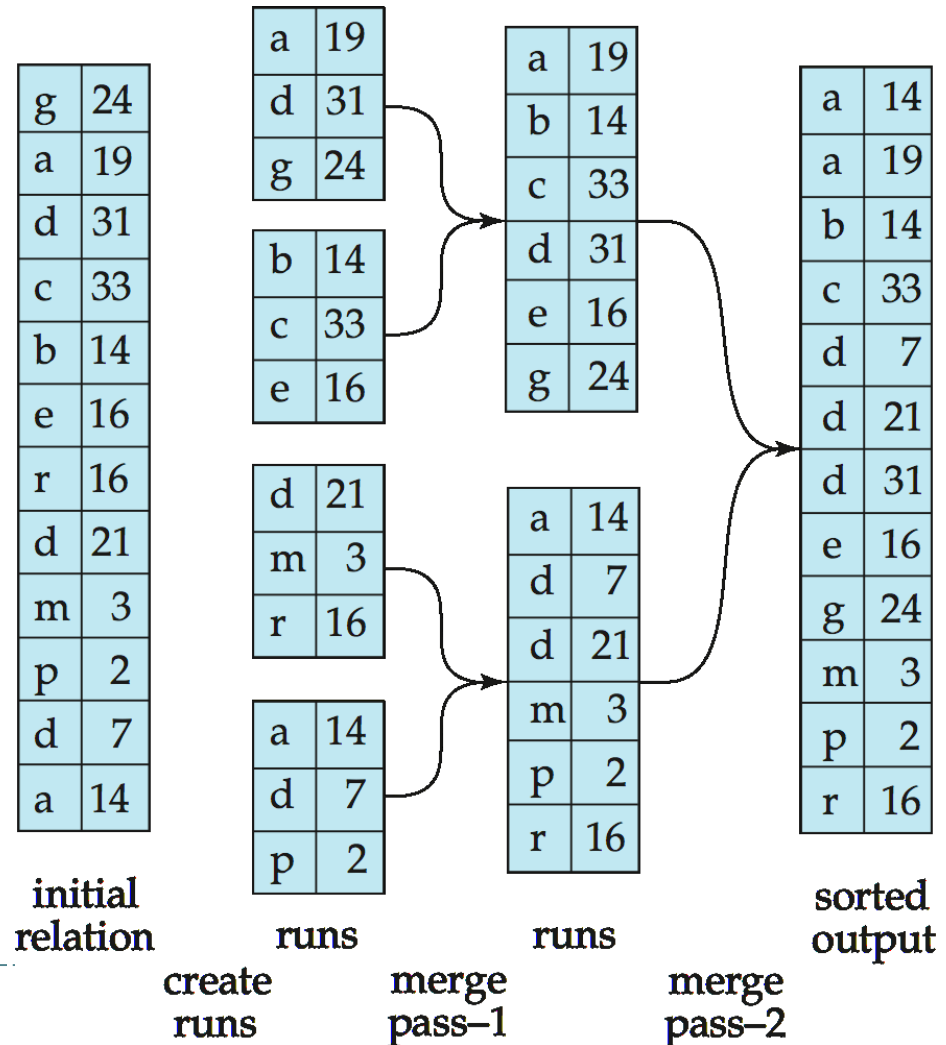
External Merge Sort: Cost analysis

► **Cost of transfers:** $b_r (2 \lceil \log_{M-1}(b_r/M) \rceil + 1)$

Applying this equation to the right example:

- $b_r = 12, M = 3$
- we get a total of $12 * (4+1) = 60$ block transfers

[Note] The above number *do not include the cost of writing out the final result.*



External Merge Sort: Cost analysis

- ▶ **Cost of seeks** (b_r denotes the *number of blocks containing records of r*)
- ▶ *Run generation* requires seeks for reading data for each of the runs as well as for writing the runs ($2\lceil b_r/M \rceil$ seeks).
- ▶ *During merge*
 - ▶ If data are read b_b blocks at a time from each run (that is, b_b buffer blocks are allocated to each run), then *each merge pass* would require around $\lceil b_r/b_b \rceil$ seeks for *reading data*.
 - ▶ Although the output is written sequentially, if it is on the same disk as the input runs the *head may have moved away* between writes of consecutive blocks.

External Merge Sort: Cost analysis

- ▶ **Cost of seeks** (b_r denotes the *number of blocks containing records of r*)
- ▶ *Run generation* requires seeks for reading data for each of the runs as well as for writing the runs ($2 \lceil b_r / M \rceil$ seeks).
- ▶ *During merge*
 - ▶ Thus we would have to add a total of $2 \lceil b_r / b_b \rceil$ seeks for *each merge pass*, except the final pass (since we assume the final result is not written back to disk).
- ▶ *The total number of seeks*

$$2 \lceil b_r / M \rceil + 2 \lceil b_r / b_b \rceil * \lceil \log_{M-1}(b_r / M) \rceil - \lceil b_r / b_b \rceil$$



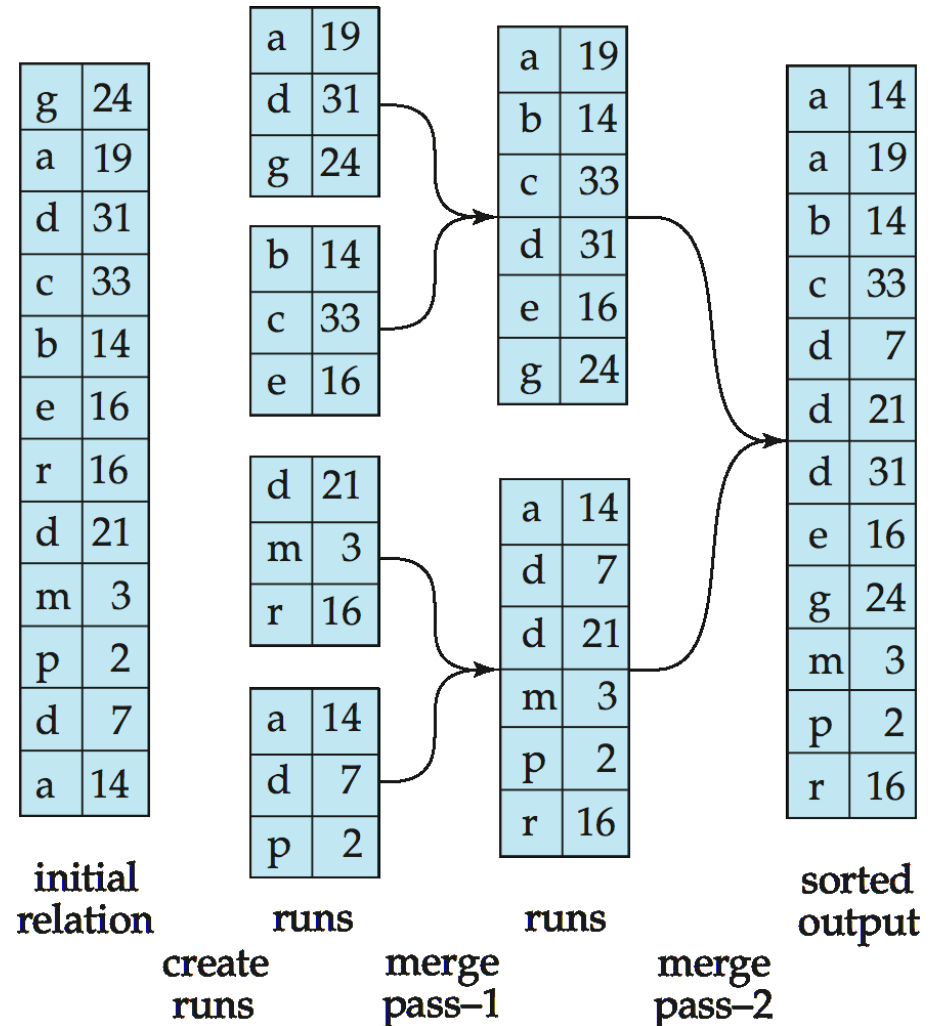
$$2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{M-1}(b_r / M) \rceil - 1)$$

External Merge Sort: Cost analysis

- **Cost of seeks:** $2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{M-1}(b_r / M) \rceil - 1)$

Applying this equation to the right example

- $B_r = 12, M = 3$
- we get a total of $8 + 12 * (2 * 2 - 1) = 44$ disk seeks.
- we set the number of buffer blocks per run, b_b to 1.



Join Operation

- ▶ There are several different algorithms to implement joins
 - ▶ *Nested-loop join*
 - ▶ *Block nested-loop join*
 - ▶ *Indexed nested-loop join*
 - ▶ *Merge-join*
 - ▶ *Hash-join*
- ▶ Here we use the term **equi-join** to refer to a join of the form: $r \bowtie_{r.A=s.B} s$
 - ▶ where A is a attribute of relation r .
 - ▶ where B is a attribute of relation s .



Join Operation

- ▶ Suppose there are two relations named *student* and *takes*.
- ▶ We use as a running example the expression:

student ⋈ *takes*

- ▶ Following information about the two relations will be used later
 - ▶ Number of records of *student*: $n_{student} = 5,000$.
 - ▶ Number of blocks of *student*: $b_{student} = 100$.
 - ▶ Number of records of *takes*: $n_{takes} = 10,000$.
 - ▶ Number of blocks of *takes*: $b_{takes} = 400$.



Nested-Loop Join

- ▶ To compute the theta join $r \bowtie_{\theta} s$, we can use algorithm

outer relation → for each tuple t_r in r do begin
inner relation → for each tuple t_s in s do begin
 test pair (t_r, t_s) to see
 if they satisfy the
 join condition θ
 if they do, add $t_r \bullet t_s$ to the result
 end
end
end

- ▶ This algorithm consists of *a pair of nested for loops*, so it is called the **nested-loop join** algorithm



Nested-Loop Join

- ▶ **Some features** of Nested-loop join algorithm
 - ▶ requires *no indices*.
 - ▶ can be used *regardless of what the join condition is*.
 - ▶ Extending the algorithm to *compute the natural join* is straightforward (only require an extra step of *deleting repeated attributes* from the tuple $t_r \cdot t_s$, before adding it to the result).
 - ▶ The nested-loop join algorithm is *expensive*, since it examines *every pair of tuples* in the two relations.

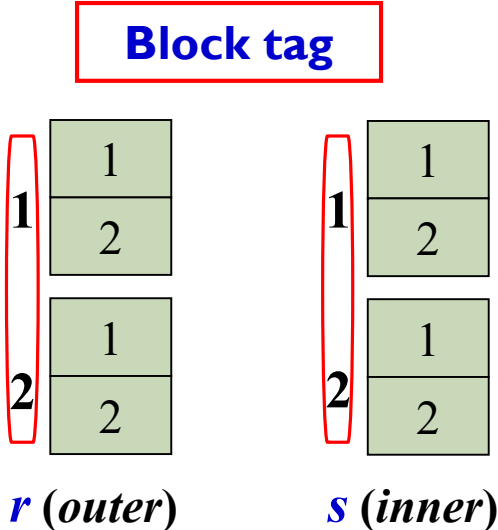


Nested-Loop Join (Cont.)

```

for each tuple  $t_r$  in  $r$  do begin
    for each tuple  $t_s$  in  $s$  do begin
        test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$ 
        if they do, add  $t_r \cdot t_s$  to the result.
    end
end
  
```

Scan order:



(1)
 $t_r(1-1)$ and $t_s(1-1)$
 $t_r(1-1)$ and $t_s(1-2)$
 $t_r(1-1)$ and $t_s(2-1)$
 $t_r(1-1)$ and $t_s(2-2)$

(3)
 $t_r(2-1)$ and $t_s(1-1)$
 $t_r(2-1)$ and $t_s(1-2)$
 $t_r(2-1)$ and $t_s(2-1)$
 $t_r(2-1)$ and $t_s(2-2)$

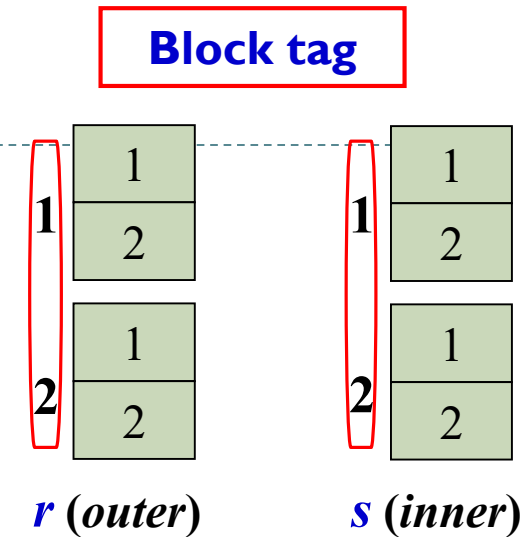
(2)
 $t_r(1-2)$ and $t_s(1-1)$
 $t_r(1-2)$ and $t_s(1-2)$
 $t_r(1-2)$ and $t_s(2-1)$
 $t_r(1-2)$ and $t_s(2-2)$

(4)
 $t_r(2-2)$ and $t_s(1-1)$
 $t_r(2-2)$ and $t_s(1-2)$
 $t_r(2-2)$ and $t_s(2-1)$
 $t_r(2-2)$ and $t_s(2-2)$



Nested-Loop Join (Cont.)

- ▶ For the algorithm, the number of *pairs of tuples* to be considered is $n_r * n_s$.
- ▶ In the *worst case*, the buffer is can hold only *one block of each relation*
 - ▶ **block transfers:** (inner) $n_r * b_s$ + (outer) b_r
 - ▶ **seeks:** (inner) $n_r * 1$ + (outer) b_r
- ▶ **Each scan** on the inner relation need *only one seek* because it is read sequentially and *a total of b_r seeks to read r .*



Nested-Loop Join (Cont.)

- ▶ In the *best case*, i.e. enough space for both relations in memory, we can *read both relations only once*

- ▶ **block transfers:** (outer) b_r + (inner) b_s

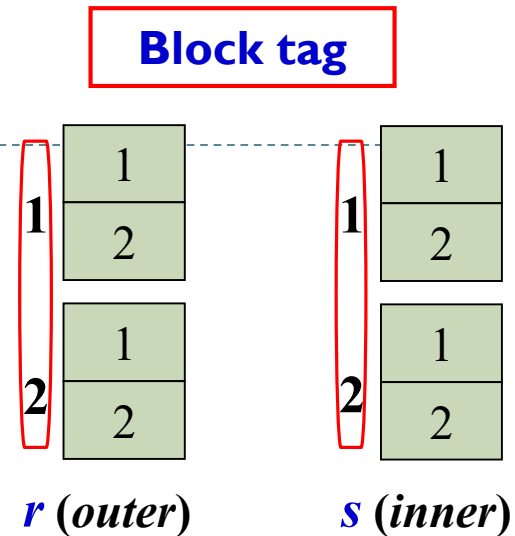
- ▶ **seeks:** (outer) 1 + (inner) 1

- ▶ If *one of the relations* (s or r) *fits entirely in memory*
 - ▶ use the *smaller relation* as the *inner relation* (if $n_r > n_s$)

- ▶ **block transfers:** (outer) b_r + (inner) b_s

- ▶ **seeks:** (outer) 1 + (inner) 1

- ▶ *same cost as that for the best case*



Nested-Loop Join (Cont.)

► **Example:** **natural join** of *student* and *takes*

- Assumption: no indices whatsoever on either relation, and use the nested loops to compute the join.

$n_{student} = 5,000, b_{student} = 100.$ t_s : seek time
 $n_{takes} = 10,000, b_{takes} = 400.$ t_T : transfer time
 $t_s = 4 \text{ ms}, t_T = 0.1 \text{ ms}$

In **worse case**: *takes* - the inner relation
The number of block transfers: $5000 * 400 + 100$
 $= 2,000,100$
The number of seeks: $5000 + 100 = 5100$
Total cost: $T_{takes} = 220410 \text{ ms}$

In **worse case**: *student* - the inner relation
The number of block transfers: $10000 * 100 + 400 = 1,000,400$
The number of seeks: $10,000 + 400 = 10,400$
Total cost: $T_{student} = 141640 \text{ ms}$

In **best case**:
Block transfers: $100 + 400 = 500$
Seeks: 2
Total cost: $T = 58 \text{ ms}$

$T_{takes} > T_{student}$

$(n_{takes} > n_{student})$

Block Nested-Loop Join

- ▶ **Block nested-loop join** is a variant of nested-loop join in which *every block of inner relation is paired with every block of outer relation*

```
for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        test pair  $(t_r, t_s)$  to see if they satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result.
      end
    end
  end
end
```

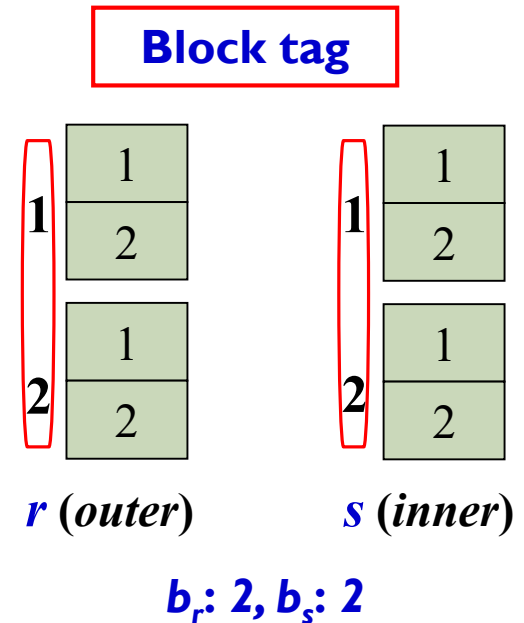
- ▶ Obtain a *major saving in block accesses*



Block Nested-Loop Join: Example

Scan order:

(1)	(3)
$t_r(1-1)$ and $t_s(1-1)$	$t_r(2-1)$ and $t_s(1-1)$
$t_r(1-1)$ and $t_s(1-2)$	$t_r(2-1)$ and $t_s(1-2)$
$t_r(1-2)$ and $t_s(1-1)$	$t_r(2-2)$ and $t_s(1-1)$
$t_r(1-2)$ and $t_s(1-2)$	$t_r(2-2)$ and $t_s(1-2)$
(2)	(4)
$t_r(1-1)$ and $t_s(2-1)$	$t_r(2-1)$ and $t_s(2-1)$
$t_r(1-1)$ and $t_s(2-2)$	$t_r(2-1)$ and $t_s(2-2)$
$t_r(1-2)$ and $t_s(2-1)$	$t_r(2-2)$ and $t_s(2-1)$
$t_r(1-2)$ and $t_s(2-2)$	$t_r(2-2)$ and $t_s(2-2)$



► In the *worst case*

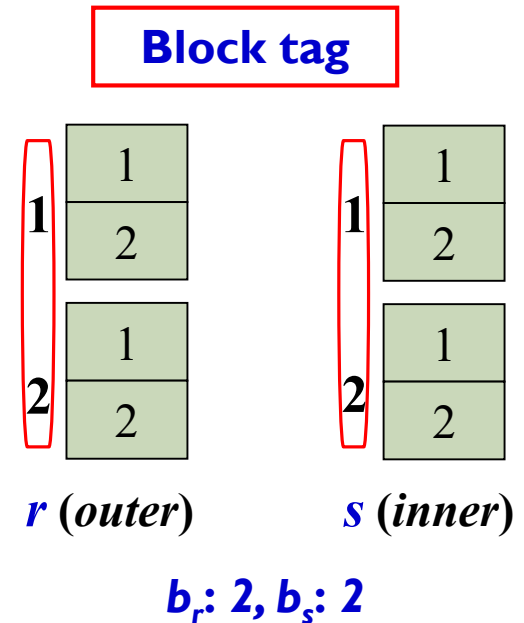
► **block transfers:** (outer) $b_r * b_s$ + (inner) b_r

► **seeks:** (inner) $b_r * 1$ + (outer) b_r

Block Nested-Loop Join: Example

Scan order:

(1)	(3)
$t_r(1-1)$ and $t_s(1-1)$	$t_r(2-1)$ and $t_s(1-1)$
$t_r(1-1)$ and $t_s(1-2)$	$t_r(2-1)$ and $t_s(1-2)$
$t_r(1-2)$ and $t_s(1-1)$	$t_r(2-2)$ and $t_s(1-1)$
$t_r(1-2)$ and $t_s(1-2)$	$t_r(2-2)$ and $t_s(1-2)$
(2)	(4)
$t_r(1-1)$ and $t_s(2-1)$	$t_r(2-1)$ and $t_s(2-1)$
$t_r(1-1)$ and $t_s(2-2)$	$t_r(2-1)$ and $t_s(2-2)$
$t_r(1-2)$ and $t_s(2-1)$	$t_r(2-2)$ and $t_s(2-1)$
$t_r(1-2)$ and $t_s(2-2)$	$t_r(2-2)$ and $t_s(2-2)$



► In the *best case* (similar to **nested-loop Join**)

► **block transfers:** (outer) b_r + (inner) b_s

► **seeks:** (outer) 1 + (inner) 1



Block Nested-Loop Join (Cont.)

- ▶ *To further improve the performance* of the nested-loop and block nested-loop procedures
 - ▶ If the *join attributes* in a natural join or an equi-join form *a key on the relation of inner relation*, then for each outer relation tuple, the inner loop can terminate on first match.
 - ▶ In block nested-loop algorithm, we can use *M - 2* disk blocks as the blocking unit for *outer relations*; use remaining *2* blocks to buffer *inner relation* and *output*.

block transfers: $b_r + \lceil b_r / (M-2) \rceil * b_s$

seeks: $\lceil b_r / (M-2) \rceil + \lceil b_r / (M-2) \rceil$

block transfers: (outer) $b_r * b_s$ + (inner) b_r
seeks: (outer) $b_r * 1$ + (inner) b_r

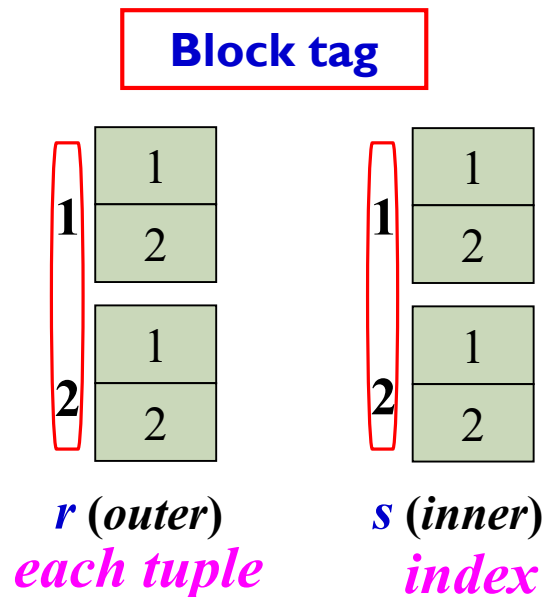
Block Nested-Loop Join (Cont.)

- ▶ *To further improve the performance* of the nested-loop and block nested-loop procedures
- ▶ *Scan inner loop alternately forward and backward*
 - ▶ This scanning method orders the requests for disk blocks.
 - ▶ So that the *data remaining in the buffer* from the previous scan can be *reused*.
- ▶ If an index is available on the *inner relation*'s join attribute, *index lookups* can replace *file scans*.



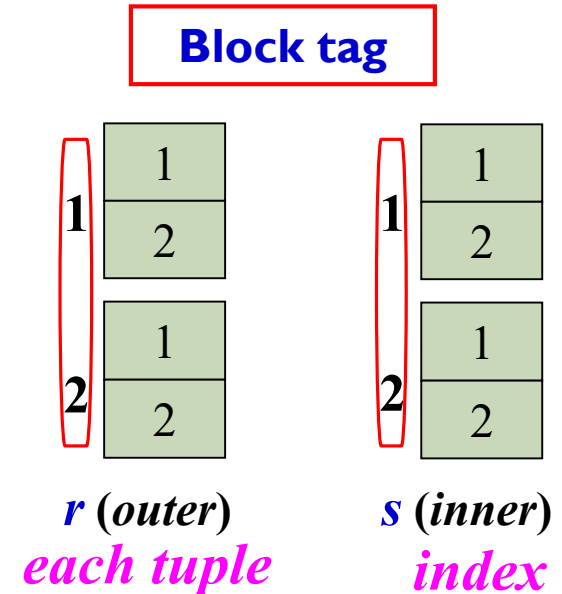
Indexed Nested-Loop Join

- ▶ For each tuple t_r in the outer relation r , use the index to look up tuples in the inner relation s that satisfy the join condition with tuple t_r .
- ▶ This join method is called an **indexed nested-loop join**.



Indexed Nested-Loop Join (Cont.)

- ▶ The *worst case* is that buffer has space for only one block of r and one block of the index (s).
- ▶ In this case, b_r I/O operations are needed to read relation r , and *each I/O need one seek and one transfer*.



- ▶ **Cost of the join:** $b_r (t_T + t_S) + n_r * c$
 - c is the *cost* of a single selection on s using the join condition.
- ▶ If indices are available on join attributes of **both r and s** , use the relation with *fewer tuples as the outer relation*.



Indexed Nested-Loop Join (Cont.)

- ▶ For example: *student* ⋈ *takes*
 - ▶ Assumptions: *takes* has a primary B⁺-tree index on the join attribute *ID*, which contains 20 entries on average in each index node.
 - ▶ Since *takes* has 10,000 tuples, **the height** of the tree is **4**.
 - ▶ One more access is needed to find the actual data.



Indexed Nested-Loop Join (Cont.)

Indexed nested-loop join

- The total disk accesses: $100 + 5000 * 5 = 25,100$
- The total cost: $T_{\text{index}} = 25,100 * 0.1 + 25,100 * 4 = 102,910 \text{ ms}$

Block nested-loop join

- The number of block transfers: $100 + 100 * 400 = 40,100$
- The number of seeks: $100 + 100 = 200$
- The total cost: $T_{\text{block}} = 40,100 * 0.1 + 200 * 4 = 4,810 \text{ ms}$

$$\begin{aligned} n_{\text{student}} &= 5,000, b_{\text{student}} = 100. \\ n_{\text{takes}} &= 10,000, b_{\text{takes}} = 400. \\ t_s &= 4 \text{ ms}, t_T = 0.1 \text{ ms} \end{aligned}$$

$$T_{\text{index}} > T_{\text{block}}$$

If we had a selection on the *student* relation that *reduces the number of rows significantly*, *indexed nested-loop join* could be significantly **faster** than *block nested-loop join*.



Merge Join

- ▶ **Merge join** can be applied to compute *natural joins* and *equi-joins*.
 - ▶ Let $r(R)$ and $s(S)$ be the relations whose natural join is to be computed.
 - ▶ Let $R \cap S$ denote their common attributes.
 - ▶ Suppose that *both relations are sorted on the attributes $R \cap S$* .
 - ▶ Then, their join can be computed by a process much like the *merge stage in the merge-sort algorithm*.



Merge Join (Cont.)

- ▶ The algorithm *associates one pointer with each relation*, and these pointers point initially to the *first tuple* of the respective relations.
- ▶ The basic steps
 - ▶ If either of the input relations r and s is not sorted on the join attributes, *sort* them first.
 - ▶ *Merge* the sorted relations to join them
 - ▶ Each time the *pointer* move through one relation
 - ▶ A group of tuples of one relation with the *same value on the join attributes* is read into S , which is a set of tuples
 - ▶ Find *matching tuples in the other relation*, and join them with tuples in S

```

pr := address of first tuple of r;
ps := address of first tuple of s;
while (ps ≠ null and pr ≠ null) do
  begin
    ts := tuple to which ps points;
    Ss := {ts};
    set ps to point to next tuple of s;
    done := false;
    while (not done and ps ≠ null) do
      begin
        ts' := tuple to which ps points;
        if (ts'[JoinAttrs] = ts[JoinAttrs])
          then begin
            Ss := Ss ∪ {ts'};
            set ps to point to next tuple of s;
          end
        else done := true;
      end
    end
    tr := tuple to which pr points;
    while (pr ≠ null and tr[JoinAttrs] < ts[JoinAttrs]) do
      begin
        set pr to point to next tuple of r;
        tr := tuple to which pr points;
      end
    end
    while (pr ≠ null and tr[JoinAttrs] = ts[JoinAttrs]) do
      begin
        for each ts in Ss do
          begin
            add ts ⋈ tr to result;
          end
        set pr to point to next tuple of r;
        tr := tuple to which pr points;
      end
    end
  end
end.

```

The algorithm requires that
every set of tuples S_s fit in main memory

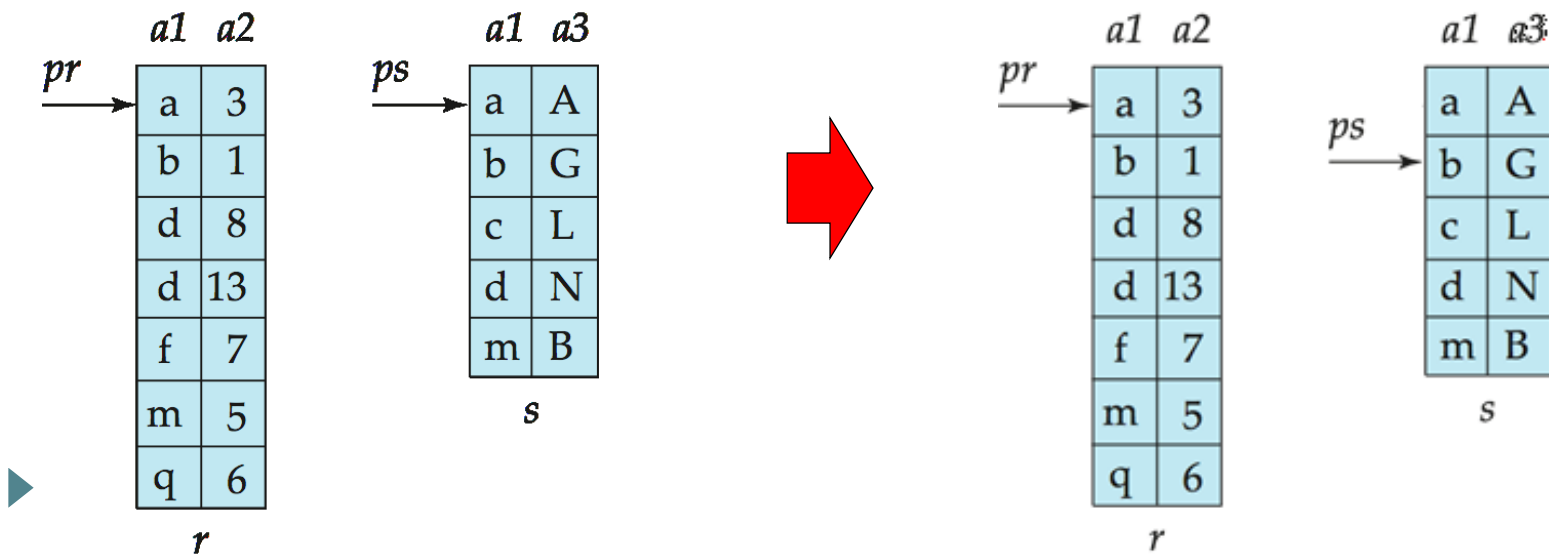
If there are some join attribute values for which S_s is larger than available memory, a **block nested-loop join** can be performed for such sets S_s .

Merge join

Merge Join: Example

► In the first pass

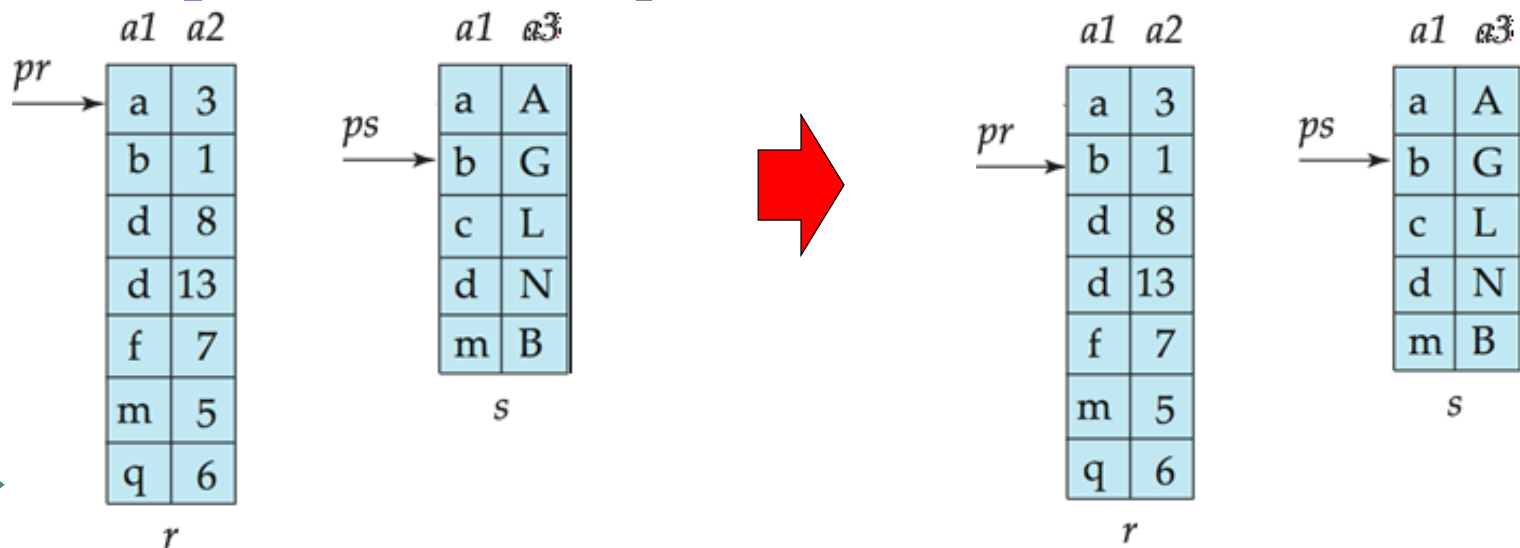
- $ps \rightarrow (a, A)$ (the first tuple in s), $t_s = (a, A)$ (the tuple ps pointed to), $S_s = \{t_s\}$, and set ps to point to next tuple of s



Merge Join: Example (Cont.)

► In the first pass

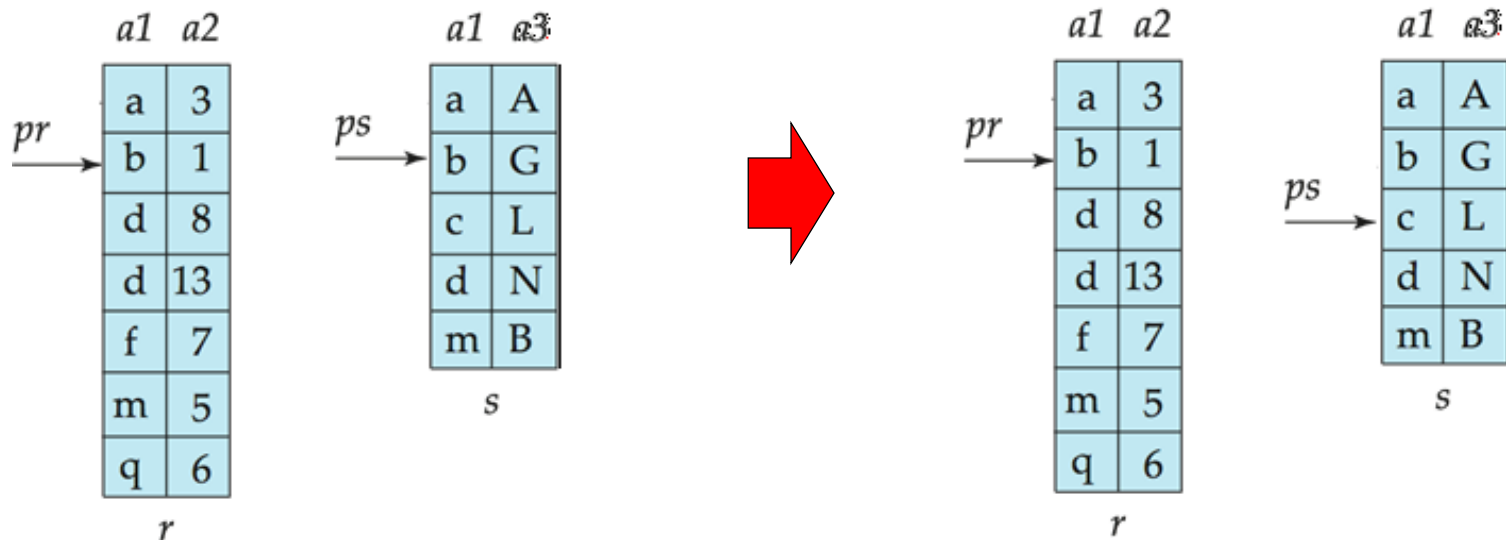
- ps moves through relation s , and put those tuples that satisfy $t_s[a1] = \text{“a”}$ condition, into S_s , which is a set of tuples. Then $S_s = \{(a, A)\}$ (only one tuple with $t_s[a1] = \text{“a”}$)
- pr moves through relation r and find the matched tuple, where $t_r[a1] = t_s[a1]$. Join it with tuples in S_s .
- Set pr to the next tuple of r .



Merge Join: Example (Cont.)

► In the second pass

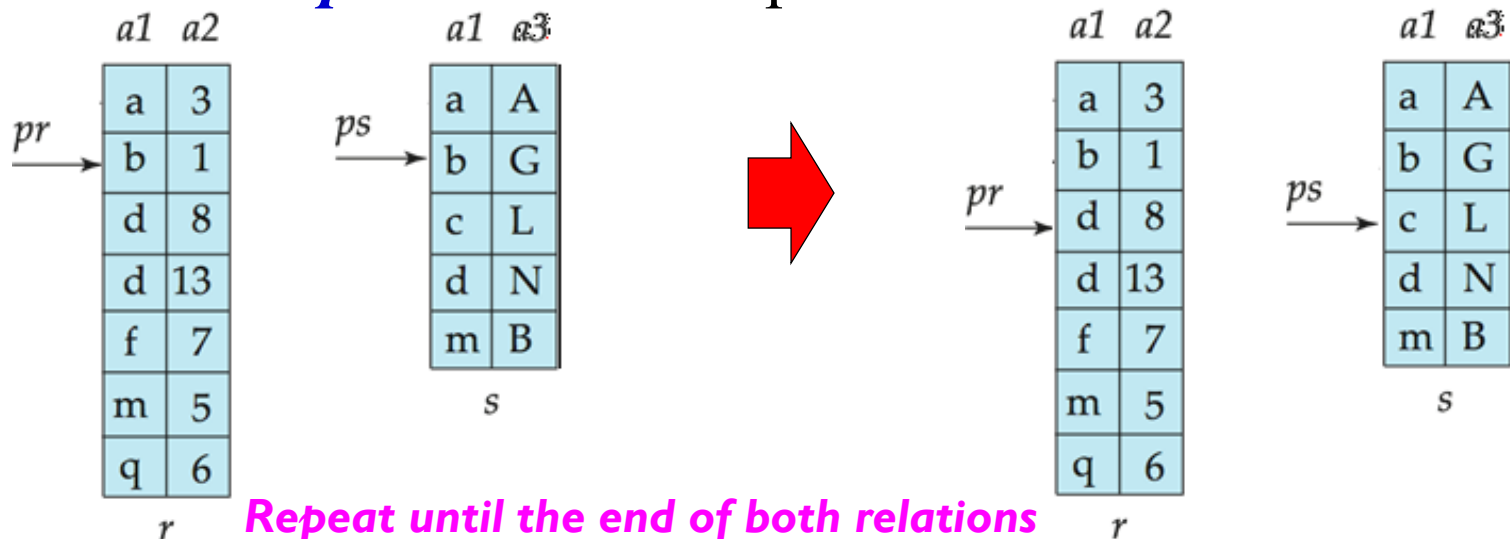
- $ps \rightarrow (b, G)$, $t_s = (b, G)$ (the tuple ps pointed to), $S_s = \{t_s\}$, and set ps to point to next tuple of s



Merge Join: Example(Cont.)

► In the second pass

- ps moves through relation s , and put those tuples that satisfy $t_s[a1] = \text{“b”}$ condition, into S_s , which is a set of tuples. Then $S_s = \{(b, G)\}$ (only one tuple with $t_s[a1] = \text{“b”}$)
- pr moves through relation r and find the matched tuple, where $t_r[a1] = t_s[a1]$. Join it with tuples in S_s .
- Set both pr to the next tuple of r .



Merge Join: Cost Analysis

- ▶ If the relations are in **sorted order**, tuples with the same value on the join attributes are in *consecutive order*.
- ▶ Each tuple in the **sorted order** needs to be *read only once*.
- ▶ Assuming all sets S_s **fit in memory**, the merge join algorithm makes only a single pass through both files, so it is **efficient**.
- ▶ The number of blocks transfer will be $b_r + b_s$.



Merge Join: Cost Analysis (Cont.)

- ▶ Assuming that b_b buffer blocks are allocated to each relation, the number of *disk seeks* required would be $\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$ disk seeks.
- ▶ If either of the input relations r and s is not sorted on the join attributes, they must be *sorted first*.
- ▶ Thus the cost of merge join is:
$$\begin{aligned} & b_r + b_s \text{ block transfers} + \\ & \lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil \text{ seeks} + \\ & \text{cost of sorting (if relations are unsorted)} \end{aligned}$$
- ▶ If some sets S_s do not fit in memory, the cost would increase slightly.

Merge Join: Cost Analysis (Cont.)

- ▶ Example: *student* ⋈ *takes*
 - ▶ The *join attribute* here is *ID*.
 - ▶ Suppose that the relations are *already sorted* on the *ID*.
 - ▶ If we assume that in the *worst case* only one buffer block is allocated to each input relation (that is, $b_b = 1$).

Block transfers	seeks	Total cost
$400 + 100 = 500$	$400 + 100 = 500$	$500 * 0.1 + 500 * 4 = 2,050$ ms

$$\begin{aligned} n_{student} &= 5,000, b_{student} = 100. \\ n_{takes} &= 10,000, b_{takes} = 400. \\ t_S &= 4 \text{ ms}, t_T = 0.1 \text{ ms} \end{aligned}$$

- ▶ In reality b_b can be set much higher since we need to buffer blocks for only two relations, and the seek cost would be significantly less.



Merge Join: Hybrid Merge-Join

- ▶ If *secondary indices* exist on both join attributes, it is possible to perform a *variation of the merge-join* operation on unsorted tuples.
- ▶ The algorithm scans the records through the indices, resulting in their being retrieved in sorted order.
- ▶ This variation presents a significant *drawback*, however, since *records may be scattered throughout the file blocks*.
- ▶ To avoid this cost, we can use a **hybrid merge-join** technique that *combines indices with merge join*.



Merge Join: Hybrid Merge-Join (Cont.)

- ▶ **Hybrid merge-join**: If one relation is sorted, and the other has a secondary B^+ -tree index on the join attribute
 - ▶ **Merge** the *sorted relation* with the *leaf entries of the B^+ -tree*.
 - ▶ The result file contains *tuples from the sorted relation* and *addresses for tuples of the unsorted relation*.
 - ▶ Then, **sort** the result file on the addresses of tuples of the unsorted relation, *allowing efficient retrieval of the corresponding tuples, in physical storage order, to complete the join*.



Hash Join

- ▶ **Hash join** is applicable for *equi-joins* and *natural joins*
- ▶ A **hash function h** is used to *partition tuples of both relations*.
- ▶ The **basic idea** is to *partition the tuples of each of the relations into sets that have the same hash value on the join attributes*.

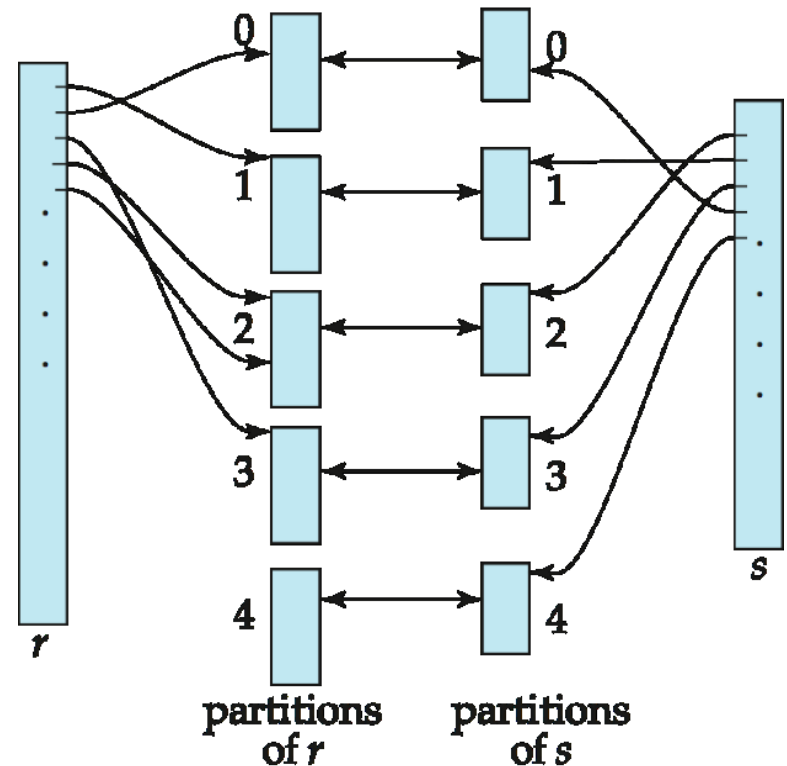


Hash Join (Cont.)

- ▶ We assume that
 - ▶ h is a hash function mapping *JoinAttrs* values to $\{0, 1, \dots, n_h\}$ (*JoinAttrs*: *common attributes* of r and s used in the natural join).
 - ▶ $r_0, r_1, \dots, r_{n_h}$ denote partitions of r tuples, each initially empty; each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[\text{JoinAttrs}])$.
 - ▶ $s_0, s_1, \dots, s_{n_h}$ denotes partitions of s tuples, each initially empty; each tuple $t_s \in s$ is put in partition s_i , where $i = h(t_s[\text{JoinAttrs}])$.
- ▶ The hash function h should have the “*goodness*” properties of *randomness and uniformity*.

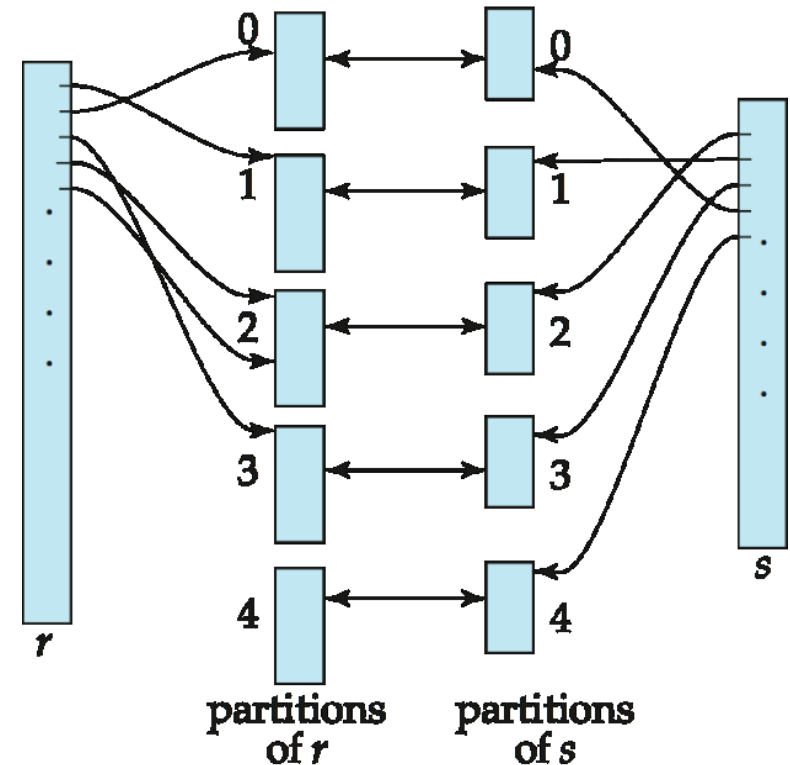
Hash Join (Cont.)

- ▶ **r tuples in r_i** need only to be compared with **s tuples in s_i** and they need not be compared with s tuples in *any other partition*, since
 - ▶ An r tuple and an s tuple that satisfy the **join condition**. They have the **same value** for the join attributes.
 - ▶ If that value is hashed to some value i , the r tuple has to be in r_i and the s tuple in s_i .



Hash Join (Cont.)

- ▶ Example: *student*, *takes*
 - *c*: a tuple in *takes*
 - *d*: a tuple in *student*
 - *h*: a hash function on the *ID* attributes of the tuples
- *d* and *c* must be tested only if $h(c) = h(d)$.
- If $h(c) \neq h(d)$, then *c* and *d* must have *different values* for *ID*.



However, if $h(c) = h(d)$, it must test *c* and *d* to see whether the values in their join attributes are the same.

Since it is possible that *c* and *d* have different *ID*s that have the same hash value.

Hash Join Algorithm

```
/* Partition s */
for each tuple  $t_s$  in  $s$  do begin
     $i := h(t_s[JoinAttrs]);$ 
     $H_{s_i} := H_{s_i} \cup \{t_s\};$ 
end
/* Partition r */
for each tuple  $t_r$  in  $r$  do begin
     $i := h(t_r[JoinAttrs]);$ 
     $H_{r_i} := H_{r_i} \cup \{t_r\};$ 
end
/* Perform join on each partition */
for  $i := 0$  to  $n_h$  do begin
    read  $H_{s_i}$  and build an in-memory hash index on it;
    for each tuple  $t_r$  in  $H_{r_i}$  do begin
        probe the hash index on  $H_{s_i}$  to locate all tuples  $t_s$ 
        such that  $t_s[JoinAttrs] = t_r[JoinAttrs];$ 
        for each matching tuple  $t_s$  in  $H_{s_i}$  do begin
            add  $t_r \bowtie t_s$  to the result;
        end
    end
end
end|
```



Hash Join Algorithm (Cont.)

- ▶ The hash-join of r and s is computed as follows
 1. **Partition the relation s** using hashing function h . When partitioning a relation, *one block of memory* is reserved as *the output buffer* for each partition.
 2. **Partition the relation r** similarly.
 3. For $i = 0, \dots, n_h$
 - (a) Read s_i and *builds a hash index* on it.
 - (b) Read the tuples in r_i from the disk *one by one*. For **each tuple t_r** , *probe* the hash index on s_i to locate **all tuples t_s** such that $t_s [\text{JoinAttrs}] = t_r [\text{JoinAttrs}]$. Output the *concatenation* of the attributes of tuples t_s and t_r .
- ▶ The relation s is the **build input**, and r is the **probe input**.

Hash Join Algorithm (Cont.)

► Note:

- The hash index on s_i is *built in memory*, it is no need to access the disk to retrieve the tuples.
- The hash function used to build this hash index must be *different from the hash function h* used earlier, but is still *applied to only the join attributes*.



Hash Join Algorithm (Cont.)

- ▶ The value n_h must be chosen to be *large enough* such that, for each i , the tuples in the partition s_i of the build relation, along with the hash index on the partition, *fit in memory*.
- ▶ It is *not necessary* for *the partitions of the probe relation to fit in memory*.
- ▶ It is best to *use the smaller input relation as the build relation*.



Recursive Partitioning

- ▶ If the value of n_h is greater than number of blocks of memory, there will *not be enough buffer blocks* to partition the relations in one pass.
- ▶ Partitioning has to be done in **repeated passes**.
 - ▶ In one pass, the input can be split into at most as many partitions as there are blocks available for use as output buffers.
 - ▶ Each bucket generated by one pass is separately read in and partitioned again in the next pass, to create smaller partitions.



Recursive Partitioning (Cont.)

- ▶ Partitioning has to be done in **repeated passes**.
 - ▶ The *hash function used in a pass* is, of course, *different from* the one used in *the previous pass*.
 - ▶ *Repeat this splitting* of the input until each partition of the build input fits in memory.
 - ▶ Such partitioning is called **recursive partitioning**.
- ▶ A relation does not need recursive partitioning if $M > n_h + 1$, or equivalently $M > (b_s/M) + 1$, which simplifies (approximately) to $M > \sqrt{b_s}$.



Handling of Overflows

- ▶ **Hash-table overflow** occurs in partition i of the build relation s if the *hash index on s_i is larger than main memory*.
- ▶ Many tuples in s with *same value* for join attributes
- ▶ *Bad hash function*: does not have the properties of randomness and uniformity.



Handling of Overflows (Cont.)

- ▶ Partitioning is said to be **skewed** if some partitions have more tuples than the average and others will have fewer.
- ▶ A *small amount of skew* can be handled by *increasing the number of partitions* so that the expected size of each partition (including the hash index on the partition) is less than the size of memory.
- ▶ The number of partitions is therefore increased by a small value, called the *fudge factor*.



Handling of Overflows (Cont.)

- ▶ **Hash-table overflows** can be handled by either *overflow resolution* or *overflow avoidance*.
- ▶ **Overflow resolution** is performed during the *build phase*
 - ▶ When overflow detected, partition s_i is further partitioned into smaller partitions by using a different hash function.
 - ▶ Similarly, r_i is also partitioned using the new hash function, and only tuples in the matching partitions need to be joined.



Handling of Overflows (Cont.)

- ▶ **Hash-table overflows** can be handled by either *overflow resolution* or *overflow avoidance*.
- ▶ **Overflow avoidance** performs the partitioning carefully to avoid overflows during *build phase*.
 - ▶ The *build relation s* is initially partitioned into many small partitions, and then some partitions are combined in such a way that each combined partition fits in memory.
 - ▶ The *probe relation r* is partitioned in the same way as the combined partitions on s , but the sizes of r_i do not matter.



Handling of Overflows (Cont.)

- ▶ **Hash-table overflows** can be handled by either *overflow resolution* or *overflow avoidance*.
- ▶ Both approaches may fail with *large numbers of tuples in s* have the same value for the join attributes.
- ▶ In this case, we can use other join techniques, such as *block nested-loop join*, on those partitions.



Cost of Hash-Join

- ▶ **CASE I:** *no hash-table overflow & no recursive partitioning*
 - ▶ The **partitioning** of the two relations r and s calls for a *complete reading* of both relations, and a subsequent *writing back* of them: **$2(b_r + b_s)$ block transfers**
 - ▶ The **build and probe phases** *read* each of the partitions once: **$b_r + b_s$ block transfers** (the number of blocks occupied by partitions could be slightly more than **$b_r + b_s$** , as a result of *partially filled blocks*)
 - ▶ **Accessing partially filled blocks** can add an overhead of **at most $2n_h$** for each of the relations (each of the n_h partitions could have a partially filled block that has to be written and read back): **$2 * 2n_h$ block transfers**

Cost of Hash-Join

- ▶ **CASE I:** *no hash-table overflow & no recursive partitioning*
 - ▶ b_b blocks for the *input buffer* and *each output buffer*
 - ▶ **Partitioning** requires $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil)$ seeks.
 - ▶ **The build and probe phases** require only **1 seek for each of the n_h partitions of each relation** (each partition can be read sequentially)
 - ▶ Hash join thus requires: $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2n_h$ seeks

Cost of hash join

$$3(b_r + b_s) + 4 * n_h \text{ block transfers} + 2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2 * n_h \text{ seeks}$$

Cost of Hash-Join

- ▶ **CASE II:** *no hash-table overflow & recursive partitioning*
 - ▶ *Each pass* reduces the size of each of the partitions by an expected factor of $M - 1$, and *passes are repeated* until each partition is of size at most M blocks.
 - ▶ So the expected number of passes required for **partitioning build relation s** is therefore $\lceil \log_{M-1}(b_s) - 1 \rceil$.
 - ▶ In each pass, every block of s is read in and written out, the total block transfers for partitioning of s is $2b_s \lceil \log_{M-1}(b_s) - 1 \rceil$.
 - ▶ The same to r : $2b_s \lceil \log_{M-1}(b_s) - 1 \rceil$.
 - ▶ The total block transfers: $2(b_r + b_s) \lceil \log_{M-1}(b_s) - 1 \rceil + b_r + b_s$.

Cost of Hash-Join

- ▶ **CASE II:** *no hash-table overflow & recursive partitioning*
 - ▶ b_b blocks are allocated for *buffering each partition*.
 - ▶ Ignoring the relatively small number of seeks during the build and probe phase, hash join with recursive partitioning requires:

$$2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) \lceil \log_{M-1}(b_s) - 1 \rceil \text{ seeks}$$

Cost of hash join

$$2(b_r + b_s) \lceil \log_{M-1}(b_s) - 1 \rceil + b_r + b_s \text{ block transfers} + 2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) \lceil \log_{M-1}(b_s) - 1 \rceil \text{ seeks}$$

Cost of Hash-Join (Cont.)

- ▶ The hash join can be improved *if the main-memory size is large*.
- ▶ If the *entire build input can be kept in main memory*, no partitioning is required (n_h can be set to 0).
- ▶ Cost estimate goes down to $b_r + b_s$ block transfers and 2 seeks.



Hybrid Hash Join (Cont.)

- ▶ Another optimization: **hybrid hash-join**
 - ▶ It is useful when memory sizes are relatively large, but not all of the build relation fits in memory.
 - ▶ In the **partitioning phase**, the hash-join algorithm needs a *minimum of one block* of memory as a *buffer for each created partition*.
 - ▶ b_b denotes the number of blocks used as a *buffer for the input and for each partition*.
 - ▶ For partitioning the two relations, a total of $(n_h + 1) * b_b$ blocks of memory are needed.



Hybrid Hash Join (Cont.)

- ▶ Another optimization: **hybrid hash-join**
 - ▶ It is useful when memory sizes are relatively large, but not all of the build relation fits in memory.
 - ▶ If memory is larger than $(n_h + 1) * b_b$, $M - (n_h + 1) * b_b$ blocks are used to *buffer the first partition of the build input relation* (that is, s_0), so that it will *not need* to be written out and read back in.
 - ▶ When partitioning r , it *does not write tuples in r_0 to disk*
 - **Instead**, as it generates them, the system uses them to *probe the memory-resident hash index on s_0* , to generate output tuples of the join.
 - After probing, the tuples can be *discarded*



Complex Joins

- ▶ Consider the following join with a conjunctive condition

$$r \bowtie_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n} s$$

- ▶ One or more of the join techniques described earlier may be applicable for joins on the *individual conditions*

$$r \bowtie_{\theta_1} s, r \bowtie_{\theta_2} s, \dots, r \bowtie_{\theta_n} s$$

- ▶ Compute the overall join by first computing the result of one of these simpler joins $r \bowtie_{\theta_i} s$.
- ▶ The *result of the complete join* consists of those tuples in the intermediate result that satisfy the remaining conditions

$$\theta_1 \wedge \dots \wedge \theta_{i-1} \wedge \theta_{i+1} \wedge \dots \wedge \theta_n$$

(can be tested as tuples in $r \bowtie_{\theta_i} s$ are being generated)

Other Operations: Duplicate Elimination

- ▶ **Duplicate elimination**: *sorting or hashing*
 - ▶ **Sorting** – duplicates will appear *adjacent to each other*, and all but one of the duplicates can be removed.
 - ▶ With *external sort–merge*, duplicates found while a *run is being created* can be removed before the run is written to disk.
 - ▶ The remaining duplicates can be eliminated *during merging*.
 - ▶ **Hashing** – duplicates will map into the *same bucket* on the basis of a hash function. A tuple is inserted only if it is not already present. Otherwise, the tuple is discarded.
 - ▶ Because of the *high cost of duplicate elimination*, SQL requires an explicit request to remove duplicates



Other Operations: Projection

► Projection

- We can implement projection easily by *performing projection on each tuple*, which gives a relation that could have duplicate records, and then removing duplicate.



Other Operations: Set Operations

- ▶ **Set operations** ($\cup$, $\cap$ and $-$): either use *variant of merge-join after sorting, or variant of hash-join*.
- ▶ Example: **set operations using hashing**
 1. *Partition* the two relations using the *same hash function*, and thereby create the partitions $r_0, r_1, \dots, r_{nh}$ and $s_0, s_1, \dots, s_{nh}$.
 2. *Process each partition i* as follows.
 - $r \cup s$
 1. Build an in-memory hash index on r_i .
 2. Add the tuples in s_i to the hash index only if they are not already present.
 3. Add the tuples in the hash index to the result.



Other Operations: Set Operations

- ▶ **Set operations** ($\cup$, $\cap$ and $-$): either use *variant of merge-join after sorting, or variant of hash-join*.
- ▶ Example: **set operations using hashing**
 1. *Partition* the two relations using the *same hash function*, and thereby create the partitions $r_0, r_1, \dots, r_{nh}$ and $s_0, s_1, \dots, s_{nh}$.
 2. *Process each partition i* as follows.
 - $r \cap s$
 1. Build an in-memory hash index on r_i .
 2. For each tuple in s_i , probe the hash index and output the tuple to the result only if it is already present in the hash index.



Other Operations: Set Operations

- ▶ **Set operations** ($\cup$, $\cap$ and $-$): either use *variant of merge-join after sorting, or variant of hash-join*.
- ▶ Example: **set operations using hashing**
 1. *Partition* the two relations using the *same hash function*, and thereby create the partitions $r_0, r_1, \dots, r_{nh}$ and $s_0, s_1, \dots, s_{nh}$.
 2. *Process each partition i* as follows.
 - *$r - s$*
 1. Build an in-memory hash index on r_i .
 2. For each tuple in s_i , probe the hash index, and, if the tuple is present in hash index, delete it from hash index.
 3. Add the tuples remaining in the hash index to the result.



Other Operations: Outer Join

► **Outer-join**: two strategies

1. *Compute the corresponding join, and then add further tuples to the join result to get the outer-join result.*
- Example: $r(R)$ and $s(S)$, evaluate $r \bowtie_{\theta} s$.
 - first compute $r \bowtie_{\theta} s$ and save that result as temporary relation q_1 .
 - Next, we compute $r - \Pi_R(q_1)$ to obtain those tuples in r that do not participate in the theta join.
 - Finally, pad each of these tuples with *null* values for attributes from s , and add it to q_1 to get the result of the outer join.



Other Operations: Outer Join

► **Outer-join**: two strategies

2. *Modify the join algorithms.*

- Example: $r(R)$ and $s(S)$, evaluate $r \bowtie_{\theta} s$.
 - Extend the nested-loop join algorithms to compute the left outer join
 - Tuples in the outer relation that do not match any tuple in the inner relation are written to the output after being padded with *null* values.



Other Operations: Aggregation

- ▶ **Aggregation operation**: the same way as duplicate elimination (*sorting or hashing*).
- ▶ The difference
 - ▶ Aggregation operation is *based on the grouping attributes*.
 - ▶ Instead of eliminating tuples with the same value for the grouping attribute, we *gather them into groups*, and apply the aggregation operations on each group to get the result.
- ▶ For aggregate functions such as **min**, **max**, **sum**, **count**, and **avg**, the cost estimate for implementing the aggregation operation is the *same as the cost of duplicate elimination*.



Evaluation of Expressions

- ▶ So far, we have seen algorithms for *individual operations*. Now we begin to consider how to evaluate an expression containing *multiple operations*.
- ▶ Alternatives
 - ▶ **Materialization**: *evaluate one operation at a time, in an appropriate order*. The result of each evaluation is *materialized* in a temporary relation for subsequent use (temporary results must be written back to disk)
 - ▶ **Pipelining**: *evaluating several operations simultaneously in a pipeline*, with the results of one operation passed on to the next, without the need to store a temporary relation.

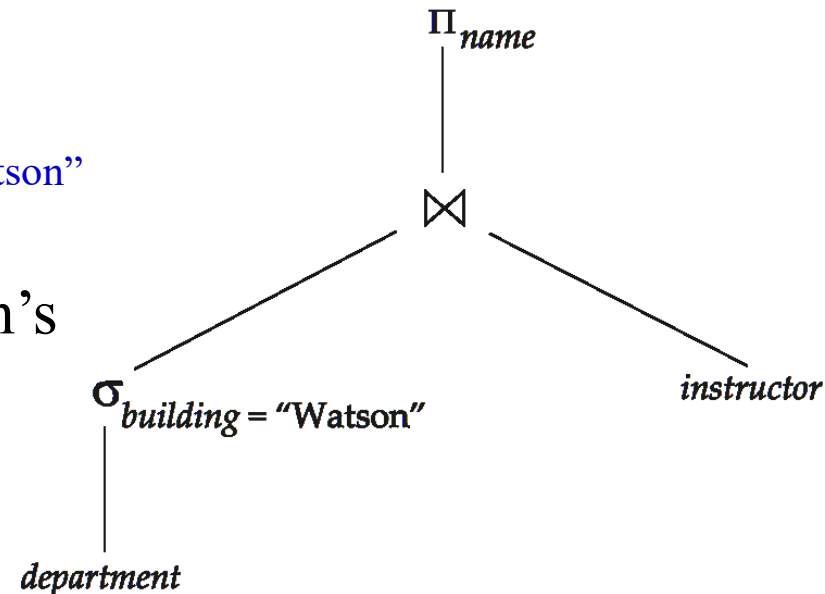


Materialization

- ▶ **Materialized evaluation**: results of each intermediate operation are created (materialized) and then are used for evaluation of the next-level operations.

$\Pi_{name} (\sigma_{building = \text{"Watson"}} (department) \bowtie instructor)$

- Build an operator tree
- First compute and store $\sigma_{building = \text{"Watson"}} (department)$ as temporary relation.
- then compute the temporary relation's join with *instructor*, and finally compute the projection on *name*.



Materialization (Cont.)

- ▶ The cost of a materialized evaluation is **NOT** simply the *sum* of the costs of the operations involved.
- ▶ Our cost formulas for operations *ignore cost of writing results to disk*, but cost of writing results to disk and reading them back can be quite high, so **Overall Cost** = *sum of costs of individual operations* + *cost of writing intermediate results to disk*.
- ▶ **Double buffering**: one continuing execution of the algorithm while the other is being written out.



Pipelining

- ▶ **Pipelined evaluation**: *evaluating several operations simultaneously in a pipeline*, with the results of one operation passed on to the next, without storing a temporary relation.
- ▶ **Benefits** of creating a pipeline of operations
 - ▶ It eliminates the cost of reading and writing temporary relations, *reducing the cost of query evaluation*.
 - ▶ It can *start generating query results quickly*, if the root operator of a query-evaluation plan is combined in a pipeline with its inputs.
- ▶ Pipelines can be executed in either of two ways: *demand-driven pipeline* & *producer-driven pipeline*

Pipelining: Demand Driven

- ▶ **Demand driven pipeline**

- ▶ *System makes repeated requests for tuples from the operation at the top of the pipeline.*
- ▶ Each time that an operation receives a request for tuples, it computes the next tuple (or tuples) to be returned, and then returns that tuple.
- ▶ If the inputs of the operation are not pipelined, the next tuple(s) to be returned can be computed from the input.
- ▶ Tuples are generated *lazily*.



Pipelining: Producer Driven

- ▶ **Producer driven pipelining**
 - ▶ *Operations do not wait for requests to produce tuples, but instead generate the tuples **eagerly**.*
 - ▶ Each operation in a producer-driven pipeline is modeled as a separate process or thread within the system that takes a stream of tuples from its pipelined inputs and generates a stream of tuples for its output.



Pipelining: Comparison

- ▶ *Producer-driven pipelining* can be thought of as *pushing* data up an operation tree *from the below*, while *demand-driven pipelining* can be thought of as *pulling* data up an operation tree *from the top*.



Homework

- 12.1** Assume (for simplicity in this exercise) that only one tuple fits in a block and memory holds at most 3 blocks. Show the runs created on each pass of the sort-merge algorithm, when applied to sort the following tuples on the first attribute: (kangaroo, 17), (wallaby, 21), (emu, 1), (wombat, 13), (platypus, 3), (lion, 8), (warthog, 4), (zebra, 11), (meerkat, 6), (hyena, 9), (hornbill, 2), (baboon, 12).
- 12.3** Let relations $r_1(A, B, C)$ and $r_2(C, D, E)$ have the following properties: r_1 has 20,000 tuples, r_2 has 45,000 tuples, 25 tuples of r_1 fit on one block, and 30 tuples of r_2 fit on one block. Estimate the number of block transfers and seeks required, using each of the following join strategies for $r_1 \bowtie r_2$:
- Nested-loop join.
 - Block nested-loop join.
 - Merge join.
 - Hash join.

